

Competitive Programming

Simone Stanganini

December 9, 2025

Contents

1	Data Structure	4
1.1	Max Heap	4
1.2	BST	4
1.3	DEQUE Double-Ended Queue	4
2	Kadane Algorithm	5
3	Sliding Window Maximum	6
3.1	BST	6
3.2	Max Heap	7
3.3	Double-Ended Queue	7
4	Binary Search for Social Distance	9
4.1	Social Distance	10
5	Tree	11
5.1	Sum	11
5.2	depth	12
5.3	BST	12
5.4	Sum u	13
5.5	MPS	14
5.6	Predecessor Problem	15
5.7	Frog And Mosquitos	16
5.8	Two Pointer Trick	18
6	Sweep Line Alogritm	19
6.1	Maximum Number of Overlapping Intervals	19

6.2	Closest Pair of Points	20
7	Prefix Sum	22
7.1	Ilya and Queries	22
7.2	Little Girl and Maximum	23
7.3	Number of Ways	24
8	Fenwick Tree	26
8.1	Counting Inversions in an Array	29
9	100 prisoners	30
10	Super clever algorithm	31
10.1	Duplicate Eleemnts in an array	31
10.1.1	log n passes	32
10.1.2	Destroy A	32
10.2	Floyds cycle finding	33
10.3	Majority Element	33
10.3.1	Boyer Moore Algorithm	34
10.3.2	Support Insertion/Deletion	34
10.4	Misra-Gries Heavy Hitter	36
11	Alice and bob	36
12	Fenwick Tree binary indexed tree, BIT	37
12.1	Dynamic Prefix Sums	37
12.2	Counting Inversion	38
12.3	Nested Segment	39
12.4	Update the Array	40
12.5	Dynamic Prefix Syms with range update	41
13	Segment Tree	42
13.1	BMQ with occurrency	43
13.2	problema piu greve	43
13.3	problema piu greveeeeeee	44
13.4	problema piu greveeeeeeeeeeeeeee	44
13.5	Triplets	45
13.6	Range update with Lazy Propagation	45

13.7 Persistent Segment Tree	47
14 Mo's Algorithm	49
14.1 Problem meno difficult	49
14.2 A Difficult Problem	50
15 (STATIC) RMQ	51
15.0.1 Cartesian Tree	53
15.1 Colored Range Query Problem	55
15.2 Range Minimum Querys	55
16 Dynamic Programming	56
16.1 Fibonacci Numbers	56
16.2 Matrix Exponentiation	56
16.3 Rod Cutting	57
16.4 facebook: Robber	57
16.4.1 shortest/longest path computation on a DAG	57
16.4.2 Road cutting: ma con il DAG	58
16.4.3 Torniamo a faceboooooook	59
16.5 Snake and ladders	59
16.6 Minimum Cost Path	60
16.7 Longest Common Subsequence LCS	60
16.8 0/1 knapsack problem	61
16.9 Fractional knapsack	62
16.10Puzzle exercise	62
16.11Longest Increasing Subsequence (LIS)	63
17 Greedy	64
17.1 Activity Selection Problem	64
17.2 Job Sequencing	65
17.3 Magic Numbers	65
17.4 Lemple-Zir	65
17.5 Lexicographically Maximum Subsequence	65
17.6 Pareto Optimal Selection	65
17.7 Woodcutter	66
17.8 queue	66
17.9 Boxes	66

1 Data Structure

1.1 Max Heap

- Estrarre il massimo $\Theta(\log |n|)$
- Vedere il massimo $\Theta(1)$
- Inserire un nuovo elemento $\Theta(\log |n|)$

Notare: **NON** si può eliminare un elemento arbitrario, inoltre è un array in memoria.

1.2 BST

Ha tutte le operazioni con costo $\Theta(\log n)$, NON può conservare elementi diversi.

Left sub tree $U \leq$ del nodo U , e sub tree di destra $>$.



- Insert(key)
- Delete(key)
- Search(key)
- Pred(key)
- Succ(key)
- Max(key)
- Min(key)

1.3 DEQUE Double-Ended Queue



E' una coda FIFO, che può avere operazioni di pop e push su entrambi i lati, per cui, è una generalizzazione della coda. Costo lineare per inserire ed eliminare. Abbiamo pop_front e push_back che sono le operazioni di una normale queue FIFO (pop e push), ma con questa possiamo fare pop_back e push_front.

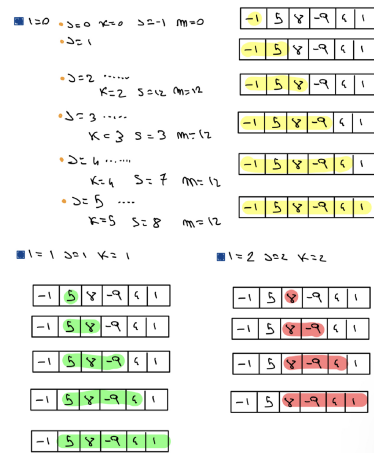
2 Kadane Algoritm

Dato un array di interi positivi e negativi, trovare il sub-array continuo con la massima somma.

```

1      /* O(n^3) */
2      fn simple_sol (v: &Vec<i32>) -> i64 {
3          let mut max: i64 = 0;
4          let len: usize = v.len();
5          for i in 0..len {
6              for j in i..len {
7                  let mut sum: i64 = 0;
8                  for k in i..(j+1) {
9                      sum = sum + v[k] as i64;
10                 }
11                 if sum > max {
12                     max = sum;
13                 }
14             }
15         }
16         max
17     }
18

```



Qua c'è una soluzione easy, dove andiamo a prendere tutti i possibili sub-array, uno alla volta, e prendiamo il massimo. `i` scorre l'array, `j` scorre da `i` a `len` per prendere tutte le posizioni e `k` fa la somma da `a i` a `j` compresa.

```

1      /* O(n^2) */
2      fn sol (v: &Vec<i32>) -> i64 {
3          let mut max: i64 = 0;
4          let len: usize = v.len();
5          for i in 0..len {
6              let mut sum: i64 = 0;
7              for j in i..len {
8                  sum = sum + v[j] as i64;
9                  if sum > max {
10                     max = sum;
11                 }
12             }
13         }
14         max
15     }
16

```

Visto che nella prima soluzione al cambiare di `j` rifacciamo sempre la solita somma con `k`, invece che ripartire da `i` ed arrivare a `j` facendo sempre le stesse operazioni, togliamo `k` e facciamo direttamente il calcolo senza rifare le stesse cose ogni volta.

```

1      /* O(n) :) */
2      fn kadane (v: &Vec<i32>) -> i64 {
3          let mut max: i64 = 0;
4          let mut sum: i64 = 0;
5          let len: usize = v.len();
6
7          for i in 0..len {
8              if sum > 0 {
9                  sum = sum + v[i] as i64;
10             } else {
11                 sum = v[i] as i64;
12             }
13             if sum > max {
14                 max = sum;
15             }
16         }
17         max
18     }
19

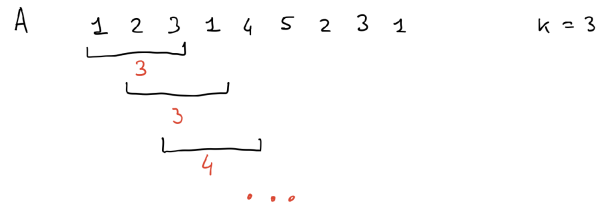
```

Adesso per raggiungere il lineare possiamo sfruttare due cose, ovvero se c'è un numero negativo. Abbiamo due proprietà da sfruttare, ovvero se ci sono numeri negativi dobbiamo toglierli, e se ci sono numeri positivi dobbiamo metterli, questo non implica il fatto che non vada messi a priori perché potremmo avere un sub array con numeri

negativi che sia maggiore di quello che avremmo non includendoli. Detto questo basta che controlliamo e quando la somma diventa negativa si riparte perché non ha senso tenerla, FINE DEL DISCORSO, solo cazzi.

3 Sliding Window Maximum

Dato un array $A[1,n]$ e un intero k , trovare l'elemento più grande di ogni finestra lunga k .



La soluzione più semplice è di prendere tutte le possibili finestre e confrontarle. Sotto il one liner del prof illeggibile per mostrare la bellezza di rust (?)

```

1  /* O(n*k) */
2  fn trivial(v: &Vec<i32>) -> Vec<i32> {
3      let mut res: Vec<i32> = Vec::with_capacity(N - K + 1);
4
5      for i in 0..(N - K + 1) {
6          let slice = &v[i..i + K];
7          let max = *slice.iter().max().unwrap();
8          res.push(max);
9      }
10
11     res
12 }
13
14 fn one_liner_trivial(v: &Vec<i32>) -> Vec<i32> {
15     v.windows(K).map(|w| *w.iter().max().unwrap()).collect()
16 }

```

Abbiamo bisogno di DS (Data Structure) per risolvere questo problema con una velocità quadratica o lineare.

Per analizzare quale sia la struttura dati migliore, andiamo a vedere le operazioni che dobbiamo fare per ogni iterazione:

- Rimuovere un elemento
- Inserire un elemento
- Restituire il massimo

3.1 BST

Target Complexity: $\Theta(n \log k)$

```

1  fn bts_sol(v: &Vec<i32>) -> Vec<i32> {
2      let mut res: Vec<i32> = Vec::with_capacity(N - K + 1);
3      let mut set = BTreeSet::new();
4      let mut max_sf = v[0]; /* max so far */
5
6      for (i, &n) in v.iter().enumerate() {
7          set.insert((n, i));
8          max_sf = max_sf.max(n);
9
10         if i >= K {
11             set.remove(&(v[i - K], i - K)); /* remove the left window number */
12             if max_sf == v[i - K] {

```

```

13         max_sf = set.last().unwrap().0; /* the bigger of the window **cost** */
14     }
15 }
16
17 if i >= K - 1 {
18     /* for the first window, and the other's */
19     res.push(max_sf);
20 }
21 }
22
23 res
24 }

```

Non potendo mettere i soliti elementi andiamo a tenerci la posizione e il valore dell'elemento nell'array all'interno dell'albero.

Dobbiamo gestire il fatto che all'inizio non siamo nemmeno alla fine della finestra, per cui teniamo per bene il massimo degli elementi che mettiamo, e quando arriviamo al momento della prima finestra vera (secondo if), e poi verrà usato sempre per mettere il massimo. L'obiettivo è tenere la finestra dentro il bst, e prendere il massimo quando serve, usando la funzione `last()`, perché ovviamente in un bst l'elemento più grande sta in fondo a sinistra. Possiamo vedere come se il massimo era l'elemento che è stato tolto dalla finestra allora si va a fare la query più costosa all'albero, se no non si fa niente perché il max è sempre `max_sf` grazie al fatto che lo aggiorniamo ogni volta.

3.2 Max Heap

Target Complexity: $\Theta(n \log n)$

```

1 fn heap_sol(v: &Vec<i32>) -> Vec<i32> {
2     let mut heap: BinaryHeap<(i32, usize)> = BinaryHeap::new();
3     let mut res: Vec<i32> = Vec::with_capacity(N - K + 1);
4
5     for i in 0..K - 1 { /* first window elements */
6         heap.push((v[i], i));
7     }
8
9     for i in K - 1..N {
10        heap.push((v[i], i));
11        /* remove all the maxes outside the window */
12        while let Some((_, idx)) = heap.peek() {
13            if *idx < i - (K - 1) {
14                heap.pop();
15            } else {
16                break;
17            }
18        }
19
20        res.push(heap.peek().unwrap().0);
21    }
22
23    res
24 }

```

Andiamo a caricare un elemento (con la posizione nell'array) per volta nella finestra, e visto che guardare il massimo nell'heap costa $\Theta(1)$, andiamo sempre a guardare il massimo, se questo è fuori dalla finestra visto che controlliamo l'index lo togliamo dall'heap e via così.

Notare come il costo sia dato dal fatto che si inserisce n volte gli elementi all'interno dell'heap, e estraiamo in caso il massimo, per cui sempre $\log n$ costo in ogni caso.

3.3 Double-Ended Queue

Si usa questa struttura, vogliamo avere sempre il massimo della window corrente nella testa della coda.

```

1 fn deque_sol(v: &Vec<i32>) -> Vec<i32> {
2     let mut q: VecDeque<usize> = VecDeque::new();

```

```

3  let mut res: Vec<i32> = Vec::with_capacity(N - K + 1);
4
5  for i in 0..K { /* first window */
6      while (!q.is_empty()) && v[i] > v[*q.back().unwrap()] {
7          q.pop_back(); /* pop al contrario */
8      }
9
10     q.push_back(i); /* normal push */
11 }
12
13 res.push(v[*q.front().unwrap()]); /* first max of the window */
14
15 for i in K..N {
16     /* remove all the elemnt outside window */
17     while !q.is_empty() && q.front().unwrap() + K <= i {
18         q.pop_front();
19     }
20
21     while (!q.is_empty()) && v[i] > v[*q.back().unwrap()] {
22         q.pop_back();
23     }
24
25     q.push_back(i);
26     res.push(v[*q.front().unwrap()]);
27 }
28
29 res
30 }

```

Dimostriamo la correttezza:

- Prop 1: q è ordinato in ordine decrescente

Un elemento è un right leader, se e se solo se, è più grande di qualsiasi altro elemento alla sua destra.

Q

x, p ₁
y, p ₂

$x < y$

- if $p_1 > p_2$,
 $\boxed{y \dots x}$
 not possible because x
 is inserted after
- if $p_1 < p_2$,
 $\boxed{x \dots y}$
 not possible
 y would kill x

4 2 3 1 1

- Prop 2: ad ogni iterazione q contiene solo right leader

① all A rightLeader cannot be removed from the window

$a > b$

② only

	b		a
--	---	--	---

↑
non right leader

So, b is not in Q because a removes it

4 Binary Search for Social Distance

```
1 fn binary_search<T: Ord>(arr: &[T], key: T) ->
  Option<usize> {
2   let mut l = 0;
3   let mut r = arr.len();
4
5   while l < r {
6     let m = l + (r - l)/2;
7
8     match key.cmp(&arr[m]) {
9       std::cmp::Ordering::Equal => return
        Some(m),
10      std::cmp::Ordering::Less  => r = m,
11      std::cmp::Ordering::Greater => l = m
        + 1,
12    }
13  }
14  None
15 }
```

Esempio di ricerca binaria fatta in rust con un metodo che usa i tipi generici, notare come `T: Ord`, serve per specificare che il tipo `T` debba avere il trait `Ord` (interfacce praticamente).

```
1 fn binary_search_smallest<T: Ord>(arr: &[T], key:
  T) -> Option<usize> {
2   let mut low = 0;
3   let mut high = arr.len();
4   let mut ans = None;
5
6   while low < high {
7     let middle = low + (high - low)/2;
8
9     match key.cmp(&arr[middle]) {
10      /* ripartiamo da sinistra per cercare
        cose più piccole*/
11      std::cmp::Ordering::Equal => {
12        ans = Some(middle);
13        high = middle
14      }
15      std::cmp::Ordering::Less  => high =
        middle,
16      std::cmp::Ordering::Greater => low =
        middle + 1,
17    }
18  }
19  ans
20 }
```

In questa implementazione andiamo a ripartire dal segmento a sinistra quando troviamo la soluzione per cercare la posizione più piccola da trovare.

```
1 fn binary_search_range<T, F>(l: T, r: T, pred: F)
  -> Option<T>
2 where /* traits */
3   T: Num + PartialOrd + FromPrimitive + Copy,
4   F: Fn(T) -> bool,
5 {
6   let mut l = l;
7   let mut r = r;
8
9   let mut ans = None;
10
11   while l < r {
12     let m = l + (r - l) / FromPrimitive::
        from_u64(2).unwrap();
13
14     match pred(m) {
15       true => {
16         l = m + T::one();
17         ans = Some(m)
18       }
19       false => r = m,
20     }
21  }
22
23  ans
24 }
```

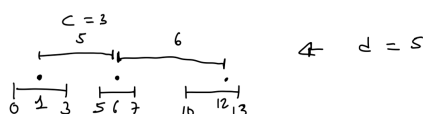
Possiamo usare la ricerca binaria per trovare il risultato di un predicato MONOTONO, per esempio dato v calcolare $\lfloor \sqrt{v} \rfloor$ usiamo la seguente funzione generalizzata.

Qua usiamo le interfacce per specificare per bene il tipo generalizzato, specifichiamo che il predicato debba essere una funzione che prende il tipo `T` e ritorna un booleano.

`T::one()` ritorna il numero che in quel tipo è l'identità ovvero 1 nel caso degli interi.

`From primitive::from_u64`, ritorna l'option di un numero u64.

Quando non troviamo la risposta andiamo a cercare a sinistra, quando invece la troviamo andiamo a destra a cercare altre risposte più grandi, questo perchè ci vogliamo avvicinare il più possibile alla risposta.



```

1 fn select_intervals(intervals: &mut Vec<(usize,
    usize)>), c: usize) -> Option<usize> {
2     /* lunghezza */
3     let l = intervals
4         .iter()
5         .fold(0, |acc, interval| acc + interval.1 -
            interval.0 + 1);
6
7     if l < c {
8         // there is no solution
9         return None;
10    }
11
12    /* non sono già in ordine? */
13    intervals.sort_unstable();
14
15    /* predicato per la ricerca binaria */
16    let pred = |d: usize| -> bool {
17        let mut last_selected = intervals[0].0;
18        let mut cnt = 1;
19        for &intervallo in intervals.iter() {
20            while intervallo.0 >= (last_selected + d) {
21                last_selected = intervallo.0;
22                cnt += 1;
23            }
24        }
25
26        cnt >= c
27    };
28
29    binary_search_range(1, l + 1, pred)
30 }

```

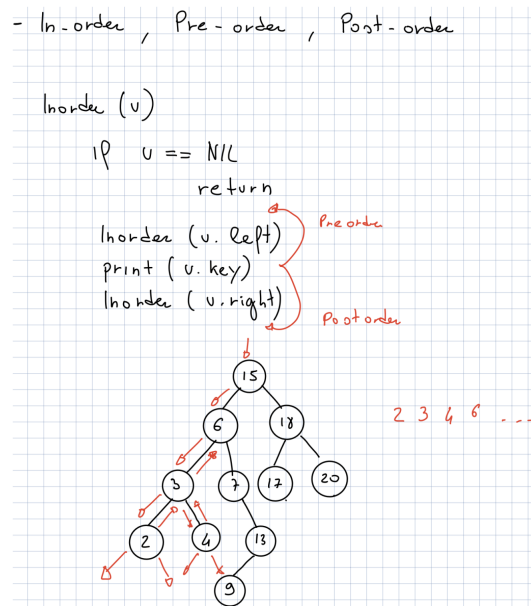
$pred(d') = \text{true}$ se e solo se è possibile trovare c punti alla distanza almeno d' .

All'interno del predicato per ogni intervallo guardiamo quanti punti possiamo metterci dentro, per cui andiamo a controllare poi in fondo se abbiamo sfiorato o pure no.

In fondo la ricerca binaria, come dall'implementazione di prima, vediamo che cerca di massimizzare il risultato l cercando quindi la distanza minima più grande.

5 Tree

Abbiamo diversi modi di fare una tree traversal:



5.1 Sum

Dato un tree T, vogliamo la somma della key di ogni nodo.

```
1  /* sum of the nodes of a tree */
2  pub fn sum(&self) -> u32 {
3      self.rec_sum(Some(0))
4  }
5
6  fn rec_sum(&self, node_id: Option<usize>) -> u32 {
7      if let Some(id) = node_id {
8          assert!(id < self.nodes.len(), "Node id is out of range");
9
10         let node = &self.nodes[id];
11         let sum_left = self.rec_sum(node.id_left);
12         let sum_right = self.rec_sum(node.id_right);
13
14         return sum_left + sum_right + node.key;
15     }
16
17     0
18 }
```

5.2 depth

Problem #2

Given a binary tree T , information moves ^{Top-down}
we want to compute $\text{depth}(v)$ (= distance from the root)

Solution

```
depth(v, d)
  if v == NIL
    return
  depth(v.left, d+1)
  print(v, d)
  depth(v.right, d+1)
```

```
1  /* print the depth of a tree */
2  fn depth (&self) -> u32 {
3      self.rec_depth(Some(0), 0);
4      0
5  }
6
7  fn rec_depth(&self, node_id: Option<usize>, d: u32) {
8      if let Some(id) = node_id {
9          assert!(id < self.nodes.len(), "Node id is out of range");
10
11          let node = &self.nodes[id];
12          self.rec_depth(node.id_left, d+1);
13          println!("depth : {d}");
14          self.rec_depth(node.id_right, d+1);
15      }
16  }
17
18 }
```

5.3 BST

Qui dobbiamo controllare che le chiavi dei nodi rispettino la definizione di BST, per cui, tutto il sotto albero di sinistra sia più piccolo del nodo, e tutto il sotto albero di destra sia più grande.

```
1  /* check if a tree is a BST */
2  pub fn check_bst(&self) -> bool {
3      self.rec_check_bst(Some(0)).b
4  }
5
6  fn rec_check_bst(&self, node_id: Option<usize>) -> Rescheck {
7      if let Some(id) = node_id {
8          let node = &self.nodes[id];
9          let res_left: Rescheck = self.rec_check_bst(node.id_left);
10         let res_right: Rescheck = self.rec_check_bst(node.id_right);
11
12         if res_left.max > node.key || res_right.min < node.key {
13             return Rescheck {
14                 b: (false),
15                 min: (min_three(node.key, res_left.min, res_right.min)),
16                 max: (max_three(node.key, res_left.max, res_right.max)),
17             };
18         } else {
19             return Rescheck {
20                 b: (res_left.b && res_right.b),
21                 min: (min_three(node.key, res_left.min, res_right.min)),
22                 max: (max_three(node.key, res_left.max, res_right.max)),
23             };
24         }
25     }
26 }
```



```

24     }
25   }
26   Rescheck {
27     b: (true),
28     min: (u32::MAX),
29     max: (u32::MIN),
30   }
31 }

```

5.4 Sum u

Dato un albero T , trovare i nodi u tali che la somma delle chiavi dei nodi nel percorso root to u , sia uguale alla somma dei nodi nel sotto albero di u .

Solution

Handwritten solution in a grid background:

```

f(u, psum) ← returns { counter, subtree sum }
if u == NIL
  return 0, 0
cl, rl = f(u.left, psum + u.key)
cr, rr = f(u.right, psum + u.key)

su = rl + rr + u.key
cu = cl + cr +
  su == psum + u.key ? 1 : 0
return cu, su

```

Implementato come lo pseudo codice, andiamo a tenerci la somma del cammino come argomento, visto che è un informazione che va dall'alto verso il basso, mentre la somma del sotto albero (compreso il nodo corrente) la restituiamo visto che è un informazione che va dal basso verso l'alto, successivamente facciamo il controllo per restituire il risultato corretto di nodi.

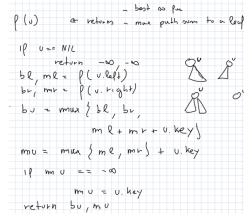
```

1  /* Given a tree , find nodes u such that the sum of the keys of the nodes in the root path to u, is
   equal to the sum of the nodes in the subtree of u. */
2  fn sum_u (&self) -> u32 {
3    self.rec_sum_u(Some(0), 0).res_counter
4  }
5
6  fn rec_sum_u(&self, node_id: Option<usize>, p_sum: u32) -> Rssum {
7    if let Some(id) = node_id {
8      assert!(id < self.nodes.len(), "Node id is out of range");
9
10     let node = &self.nodes[id];
11     let res_l = self.rec_sum_u(node.id_left, p_sum + node.key);
12     let res_r = self.rec_sum_u(node.id_right, p_sum + node.key);
13     let su = res_l.sub_sum + res_r.sub_sum + node.key;
14
15     let mut res_c = 0;
16     if su == (p_sum + node.key) {
17       res_c = 1;
18     }
19     let cu = res_l.res_counter + res_r.res_counter + res_c;
20
21     return Rssum {
22       sub_sum: (su),
23       res_counter: (cu),
24     }
25   }
26
27   Rssum {
28     res_counter: (0),
29     sub_sum: (0),
30   }
31 }

```

5.5 MPS

Dato un albero dove ogni nodo ha un valore, trovare il percorso di costo massimo da una foglia all'altra.



```

1 fn mps (&self) -> u32 {
2     self.rec_mps(Some(0)).bsf as u32
3 }
4
5 fn rec_mps(&self, node_id: Option<usize>) -> Mps {
6     if let Some(id) = node_id {
7         assert!(id < self.nodes.len(), "Node id is out of range");
8         let node = &self.nodes[id];
9         let res_l = self.rec_mps(node.id_left);
10        let res_r = self.rec_mps(node.id_right);
11        let bu = max_threei(res_l.bsf,
12                            res_r.bsf,
13                            if (res_l.mps == MIN) || (res_r.mps == MIN) {MIN}
14                            else {res_l.mps + res_r.mps + (node.key as i32)});
15
16        let mut mu = if (res_l.mps == MIN) && (res_r.mps == MIN) {MIN}
17        else {cmp::max(res_r.mps, res_l.mps) + (node.key as i32)};
18
19        if mu == MIN {
20            mu = node.key as i32;
21        }
22
23        return Mps {
24            bsf: (bu),
25            mps: (mu),
26        }
27    }
28
29    Mps {
30        bsf: (MIN),
31        mps: (MIN),
32    }
33 }

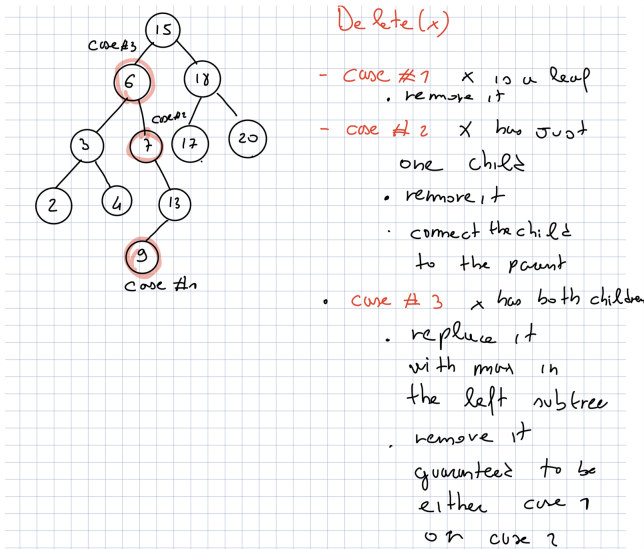
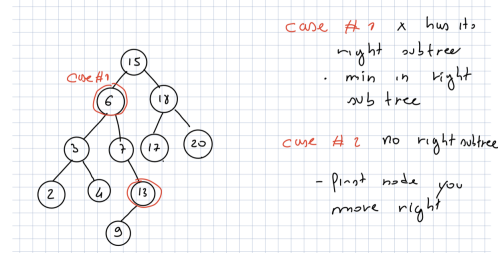
```

5.6 Predecessor Problem

Dato un insieme S di chiavi, vogliamo dare le seguenti possibilità:

- Insert (x)
- Delete (x)
- Lookup (x) is x in S ?
- Min/Max
- Predecessor (x) $\max z \parallel z \in S \text{ and } z \leq x$
- Successor (x) $\min z \parallel z \in S \text{ and } z \geq x$

Successor (x)



Se un nodo è una foglia si rimuove e basta. Se un nodo ha almeno un solo figlio si elimina e si connette il figlio al posto suo. Se un nodo a 2 figli si elimina, si cambia con il massimo nel suo sub tree di sinistra, Per il successore se se abbiamo il sottoalbero di sinistra, è il min del right subtree. Oppure come nel caso del nodo numero 13, il suo successore è 15, e vediamo come nel caso senza sottoalbero destro, si usa il padre, e scorriamo il padre fino a che non troviamo un padre che sia il figlio sinistro del suo padre, così sappiamo che sia il suo successore.

5.7 Frog And Mosquitos

Target: $\Theta((n + m) \cdot \log(n + m))$

n rane nell'asse delle x :

Ogni rana i ha un valore t_i che è la lunghezza della sua lingua, m mosche iniziano a posarsi sull'asse delle x , ogni mosca j si posa nella posizione b_j e ha un valore di a_j che è la sua grandezza.

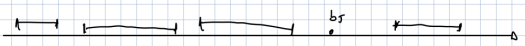
Una rana i mangia una j :

1. Se la mosca b_j è dentro la distanza t_j da x_i
 2. Se la rana i mangia la mosca j allora $t_i = t_i + a_j$
 3. Se due o più rane possono mangiare una mosca, allora la mangia la più distante
- Input: Prima riga contiene n e m ovvero quante rane e quante mosche, successivamente ci sono n righe, ognuna la posizione i dellaesima rana x_i e la lunghezza della sua lingua t_i , successivamente ci sono m righe contenenti la posizione b_j e la grandezza della mosca a_j .
 - Output: Scrivere n righe, dove ogni riga ha quante mosche ha mangiato la i esima rana e quanto la sua lingua è lunga.

Molto difficile: una soluzione sarebbe di usare un BST, si va a calcolare quanto

Solution

- Store x_i in a BST with t_i



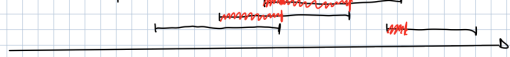
- compute $\text{predecessor}(b_j) = x_i$
check if $b_j \in [x_i, x_i + t_i]$
- Each query is solved in $\Theta(\log n)$ time

Andiamo a prendere le rane x_i , le mettiamo in un bst, per ogni mosca b_j andiamo a calcolare il predecessore(b_j), ma questo non ci porta al risultato vero visto che la lingua delle rane è dinamica.

Per risolvere questi problemi andiamo a fare 3 step:

1. Issue 1:

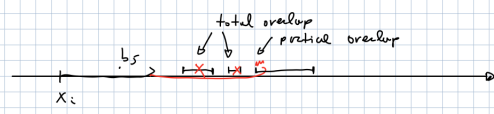
- Preprocess segments (at the beginning) to remove overlaps



Togliamo i segmenti che vanno a sovrapporsi.

2. Issue 2:

- Segments change dynamically so we may have overlaps



three possible cases for x_j

- total overlap: remove x_j and move to x_{j+1}
- partial overlap: modify x_j and STOP
- no overlap: STOP

Visto che la grandezza delle lingue è dinamica dobbiamo andare a gestire 3 casi diversi.

3. Issue 3:



- Maintain uneaten mosquitos in a BST

- When we increase tongue of frog x_i

now its $[x_i, x_i + t_i + a_n]$

Search for the successor of x_i
and check if the resulting
mosquito b_j is $\leq x_i + t_i + a_n$

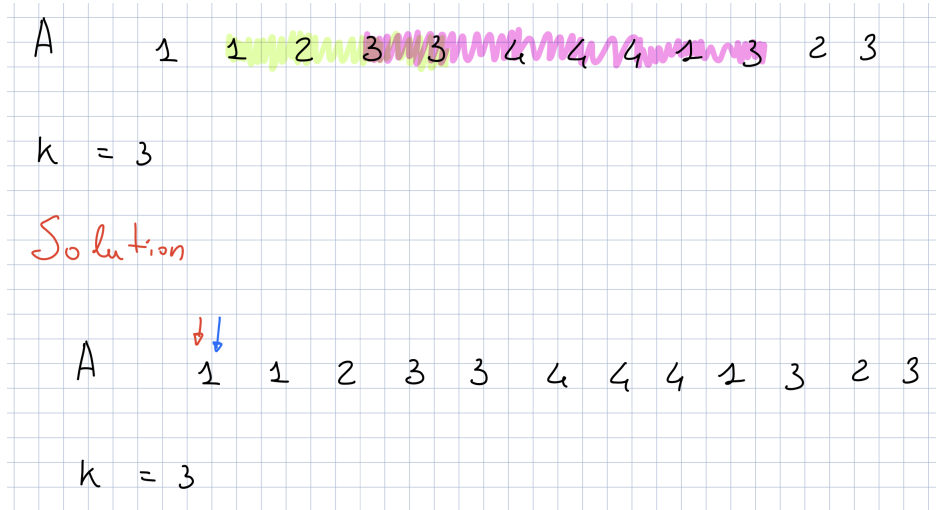
$\Theta(m \cdot \log m)$

Molte mosche potrebbero non essere mangiate per un po' visto che le rane non è detto che già da subito abbiano la lunghezza della lingua sufficiente a mangiare tutte le mosche.

5.8 Two Pointer Trick

Proble: **Longest k-good Segment** Dato un array $A[1, n]$ di valori e un valore k , un sub'array è chiamato k-good se contiene al massimo k valori differenti. Trovare il più grande k-good segment.

Si usano due puntatori per rappresentare una finestra di lunghezza variabile.



Si usano due puntatori per tenere traccia della window corrente di lunghezza variabile, in base a se la finestra è valida oppure no, spostiamo la posizione dei puntatori. Andiamo grazie ad un BST a tenere conto di quante occorrenze ci sono degli elementi nella finestra. Il primo puntatore sta all'inizio e il secondo sta alla fine. Spostiamo il puntatore a sinistra quando la finestra non è valida, spostiamo il puntatore di destra quando la finestra è valida.

(il prof dice che è talmente semplice che non servono nemmeno le note, ma comunque non ci ho capito un sega e lui non ha spiegato un cazzos) Visto che ci devono essere nella finestra solo 2

Time complexity: $\Theta(n \log k)$ con un BST, $\theta(n^k)$ con un array, $\theta(n)$ con hashing, quindi se dobbiamo tenere conto del tempo si usa un array.

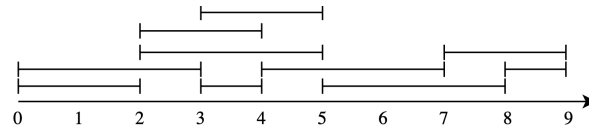
6 Sweep Line Alogritm

E' un algoritmo usato per risolvere problemi nella retta e nel piano.

6.1 Maximum Number of Overlapping Intervals

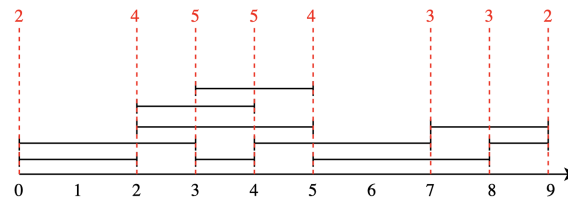
Ci viene dato un set di n intervalli $[s_i, e_i]$ su una linea. Diciamo che un due intervalli $[s_i, e_i]$ e $[s_j, e_j]$ si sovrappongono se e solo se la loro intersezione non è il set vuoto, ovvero se esiste almeno un punto x che appartiene a entrambi.

L'obbiettivo è di calcolari il massimo numero di intervalli che si sovrappongono



L'algoritmo si basa sul fatto di tenere questa **sweep line**, che è solamente una linea immaginaria verticale, e ci manteniamo via via lo stato dell'algoritmo.

Questa linea la muoviamo solo in determinati punti delle x , solo dove potremmo avere dei punti di maggior interesse, come fa la linea nella figura seguente:



Notare come per esempio nel punto 1 o 6 non andiamo.

```
1 #[derive(PartialOrd, Ord, PartialEq, Eq, Debug)]
2 enum Event {
3     Begin,
4     End,
5 }
6
7 pub fn max_overlapping(intervals: &[(usize, usize)]) -> usize {
8
9     println!("intervals {:?}", intervals);
10
11     let mut pairs: Vec<_> = intervals
12         .iter()
13         .flat_map(|&(b, e)| [(b, Event::Begin), (e, Event::End)])
14         /*è una map che appiattisce i risultati di una map normale,
15         infatti in questo caso restituiamo per ogni elemento che troviamo,
16         le coppie di usize, un array con le coppie con end e begin,
17         in questa maniera con flatmap e collect si crea un array unico.*/
18         .collect();
19
20     pairs.sort_unstable();
21     /* sorting per ordinare le coppie in base al primo valore come sempre */
22
23     pairs
24         .into_iter() //
25         .scan(0, |counter, (_, kind)| {
26             if kind == Event::Begin {
27                 *counter += 1;
28             } else {
29                 *counter -= 1;
30             }
31         })
```

```

31         Some(*counter)
32     })
33     /*lo scan è come una map ma si porta dietro un conter, e se usiamo collect,
34     * restituisce tanti valori, mentre in questo caso si usa per trovare il massimo
35     * tra tutti questi che restituisce */
36     .max()
37     .unwrap()
38 }

```

In questa implementazione andiamo a creare un array di coppie, con ogni coppia l'inizio e la fine di ogni segmento (usando l'enum), dopo andiamo ad ordinarli in ordine crescente, dopo di che con la scan andiamo a tenere un contatore che decrementa quando troviamo un end e cresce quando troviamo un begin, in questa maniera andiamo a tenere il conto sempre del numero corrente dei segmenti in quello specifico punto della retta (in questo caso sempre o l'inizio o la fine di un segmento) e restituiamo solamente il massimo.

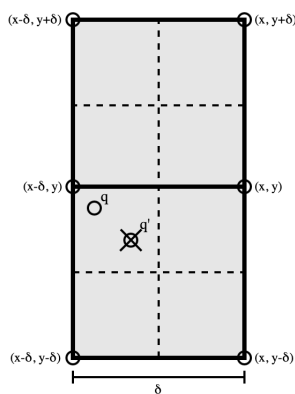
6.2 Closest Pair of Points

Un set di n punti nel piano, l'obiettivo è di trovare la coppia di punti più vicina.

La distanza tra due punti è $d((x_1, y_1), (x_2, y_2)) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

Un algoritmo brute force ci mette $\Theta(n^2)$

Un algoritmo più veloce sfruttando la sweep line, sortiamo in ordine crescente i punti in base alla x , teniamo conto della distanza più piccola chiamandola δ .



Mettiamo caso che abbiamo una δ , sicuramente se andiamo a prendere un punto (x, y) , tutti i punti che sono più vicino a x di δ avranno sicuramente le coordinate comprese tra $[x - \delta, x]$ e $[y - \delta, y + \delta]$.

L'algoritmo tiene tutti i punti che servono ordinati per le y in un BST, il quale all'inizio è vuoto, processiamo uno alla volta $p = (x, y)$, e per ogni p all'interno del set (BST) andiamo a iterare sui punti che hanno la coordinata y tra $[y - \delta, y + \delta]$, e rimuoviamo quelli dentro a questo intervallo con la coordinata x minore di $x - \delta$,

```

1 pub fn distance_squared(p: (i64, i64), q: (i64, i64)) -> i64 {
2     (p.0 - q.0).pow(2) + (p.1 - q.1).pow(2)
3 }
4
5 use std::collections::BTreeSet;
6 use std::ops::Bound::Included;
7
8 // Returns the (squared) Euclidean distance between the closest pair of
9 // points in 'points'
10 pub fn closest_pair(points: &mut [(i64, i64)]) -> Option<i64> {
11     if points.len() < 2 {
12         return None;
13     }
14
15     points.sort_unstable_by_key(|p| (p.1, p.0)); // sort by y
16
17     let min_y = points[0].1;
18     let max_y = points.last().?.1;
19
20     /* first delta */

```



```

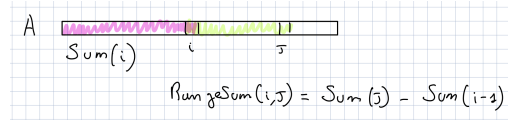
21 let mut delta = distance_squared(points[0], points[1]);
22
23 let mut set: BTreeSet<(i64, i64)> = BTreeSet::new();
24 for &point in points.iter() {
25     /* We remove all the point in the set that have the y in [y-\delta,y+\delta]
26      * and have the x coordinate smaller than x-\delta
27      */
28     let to_delete: Vec<_> = set
29         .range((
30             Included(&(point.0 - delta, min_y)),
31             Included(&(point.0 + delta, max_y)),
32         ))
33         .filter(|p| p.0 - delta >= point.0)
34         .cloned()
35         .collect();
36
37     // Remove those points
38     for p in to_delete {
39         set.remove(&p);
40     }
41
42     /* Search again in the same point that have the y in the [y-\delta,y+\delta]
43      * and we compute the distances with survived points.
44      * Update delta if needed */
45     delta = set
46         .range((
47             Included(&(point.0 - delta, min_y)),
48             Included(&(point.0 + delta, max_y)),
49         ))
50         /* fold is like scan but return only one result, and in this case we initialize the
51          * counter with delta, for take the minimum
52          */
53         .fold(delta, |acc, &p| acc.min(distance_squared(point, p)));
54
55     set.insert(point);
56 }
57
58 Some(delta)
59 }

```

7 Prefix Sum

Dato un array $A[1, n]$ di interi, vogliamo supportare le seguenti operazioni:

- $\text{sum}(i) = \sum_{k=1}^i A[k]$
- $\text{RangeSum}(i, j) = \sum_{k=i}^j A[k]$



Per risolvere questo usiamo l'array P dove $P[i] = \sum_{k=1}^i A[k]$, in questa maniera la query di **sum** viene risolta di $\Theta(1)$.

A	1	2	3	4	5	6	7	8
	2	4	1	7	3	0	4	2
P	2	6	7	14	17	17	21	23

Per calcolare per esempio il $\text{RangeSum}(j, j) = P[i] - P[i - 1]$, e infatti per esempio $\text{RangeSum}(2, 6) = P[6] - P[1] = 17 - 2 = 15$

```

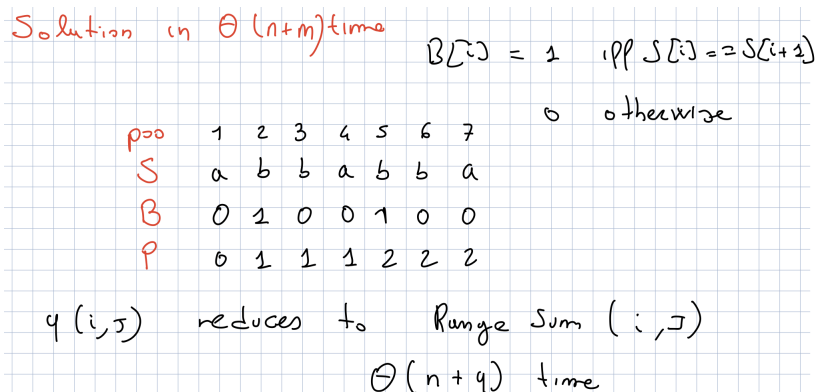
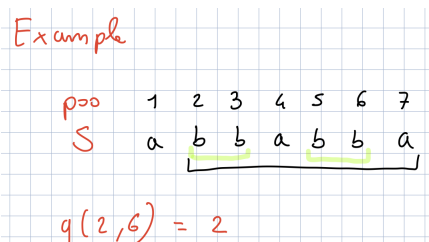
1 let a = vec![2, 4, 1, 7, 3, 0, 4, 2];
2
3 let psums = a
4   .iter()
5   .scan(0, |sum, e| {
6     *sum += e;
7     Some(*sum)
8   })
9   .collect::<Vec<_>>();
10
11 assert!(psums.eq(&vec![2, 6, 7, 14, 17, 17, 21, 23]));

```

7.1 Ilya and Queries

Data una stringa binaria $S[1, n]$, che consiste sono il **a** e **b**, e un insieme di m queries, allora:

$q(i, j)$ = ritorna il numero di posizioni $k \in [i, j]$ tali che $S[k] = S[k + 1]$



Faccio solo uno screen, tanto è il range sum applicato a questo problema, non capisco perché l'array B ma comunque conta solo se la q è rispettata.

```

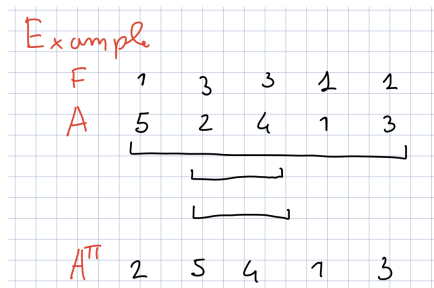
1 struct Ilya {
2     psums: Vec<usize>,
3 }
4
5 impl Ilya {
6     pub fn new(s: &str) -> Self {
7         let psums = s
8             .as_bytes()
9             .windows(2)
10            .map(|w| if w[0] == w[1] { 1usize } else { 0usize })
11            .scan(0, |sum, e| {
12                *sum += e;
13                Some(*sum)
14            })
15            .collect::<Vec<_>>();
16
17         Self { psums }
18     }
19
20     pub fn q(&self, i: usize, j: usize) -> usize {
21         assert!(i < j);
22         assert!(j <= self.psums.len());
23
24         self.psums[j - 1] - if i != 0 { self.psums[i - 1] } else { 0 }
25     }
26 }

```

7.2 Little Girl and Maximum

Dato un array $A[1, n]$ e un set Q di q queries, ogni q è una range sum queries da i a j e ritorna la somma di $A[i..j]$

L'obiettivo è quello di permutare i valori di A per massimizzare la somma dei risultati delle query di Q .



Notare: un'idea iniziale è assegnare i valori più grandi nelle posizioni accedute da più query possibili, quindi ordinare in ordine decrescente, e ordinare gli index di A in ordine decrescente in base al numero di accessi a una determinata posizione, questa cosa costa $\Theta(n \log n)$ dopo che abbiamo computato le frequenze negli index, il problema è fare questa cosa.

Ovvero costruire l'array $F[1, n]$ dove $F[i]$ è il numero di volte che l'index i appartiene ad una query di Q .

Potremmo usare lo sweep line algorithm per fare questo, visto che le query sono dei segmenti, e calcolare le frequenze si basa sul calcolare quanti segmenti sono in overlap su quel singolo punto basterebbe usare l'algoritmo discusso precedentemente, il che avrebbe costo $\Theta(q \log q)$.

Esiste tuttavia una soluzione che sfrutta la prefix sum, si basa tutto sul costruire un array $U[1..n]$ tale che le sue prefix sum sono uguali al nostro array target Fr .

Inizialmente tutto U è $= 0$, per una query (l, r) si aggiunge facciamo $U[l] += 1$ e $U[r + 1] -= 1$.

In questa maniera la prefix sum di $U[i]$ rispetta la frequenza degli index di F , perché la prefix sum rimane invariata per indici inferiori di l , aumentata di 1 per indici compresi tra l e r , e di nuovo invariata per indici maggiori di r .

```

1 // We assumes queries are 0-based indexed
2 pub fn little_girl(a: &[i64], q: &[(usize, usize)]) -> i64 {
3     if a.is_empty() {
4         return 0;
5     }
6
7     let mut u = vec![0i64; a.len()];
8
9     for &(l, r) in q {
10        assert!(l <= r);
11        assert!(r < u.len());
12
13        u[l] += 1;
14        if r + 1 < u.len() {
15            u[r + 1] -= 1;
16        }
17    }
18
19    let mut f = u
20        .iter()
21        .scan(0, |sum, e| {
22            *sum += e;
23            Some(*sum)
24        })
25        .collect::<Vec<_>>();
26
27    // we sort both f and a in decreasing order, nothing changes
28    f.sort_unstable();
29    let mut a_sorted = a.to_vec();
30    a_sorted.sort_unstable();
31
32    a_sorted
33        .iter()
34        .zip(f)
35        .fold(0, |result, (value, freq)| result + value * freq)
36 }

```

7.3 Number of Ways

Dato $A[1..n]$, contare quante volte possiamo splittare A in 3 parti tali che la somma di queste tre parti è la stessa tra tutte e tre.

Chiamiamo S la somma di tutti i valori di A , se sicuramente se S non è divisibile per 3 la soluzione non esiste.

Creaiamo una array C che tiene nella posizione i esima il numero dei valori (PARTENDO DA N, QUINDI AL CONTRARIO, SUFFISSI VUOL DIRE LA PARTE FINALE DIO IMPIEGATO), dentro $A[i..n]$ che hanno la loro somma con $S/3$.

Dopo di che da sinistra a destra facciamo al predix sum di A , e ogni volta che la prefix sum nella posizione i è uguale a $S/3$ andiamo a sommare $C[i + 2]$.

Questo perchè la parte $A[1..i]$ a somma $S/3$ e può essere combinato con qualsiasi coppia della parte $A[i + 1..n]$ tanto tutti e due hanno la somma $S/3$. Visto che $A[i + 1..n]$

```

1 pub fn number_of_ways(a: &[i64]) -> usize {
2     let sum: i64 = a.iter().sum();
3
4     if sum % 3 != 0 {
5         return 0;
6     }
7
8     let target = sum / 3;
9     let mut c: Vec<_> = a
10        .iter()
11        .rev()
12        .scan(0, |sum, e| {
13            *sum += e;
14            Some(*sum)
15        })

```

```

16     .scan(0, |counter, sum| {
17         if sum == target {
18             *counter += 1usize
19         };
20         Some(*counter)
21     })
22     .collect();
23
24     c.reverse();
25
26     let mut result = 0;
27     let mut sum = 0;
28     for (i, &v) in a[..a.len() - 2].iter().enumerate() {
29         sum += v;
30         if sum == target {
31             result += c[i + 2];
32         }
33     }
34
35     result
36 }

```

8 Fenwick Tree

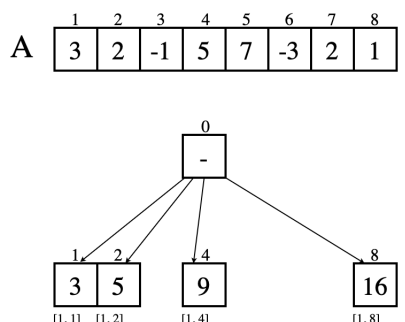
Dato un array $A[1, n]$ vorremmo supportare le seguenti operazioni:

- $\text{Sum}(i) = \sum_{k=1}^i A[k]$
- $\text{Add}(i, v) = A[i] + v$

Possiamo provare con un pò di soluzioni inutili, tipo usare un array brutto, e quindi la $\text{Sum}(i)$ richiede $\Theta(n)$ e invece $\text{add}(i, v)$ richiede $\Theta(1)$, mentre se usassimo un array di prefix sum di A , allora sarebbe il contrario, la somma chiederebbe $\Theta(1)$ tempo mentre la add chiederebbe $\Theta(n)$.

Notare: con il fenwick Tree si fanno tutte queste operazioni in $\Theta(\log n)$, notare inoltre che il fenwick tree è una data structure **implicita**, questo perché richiede solamente $\Theta(1)$, di spazio per essere oltre all'array A per essere usata.

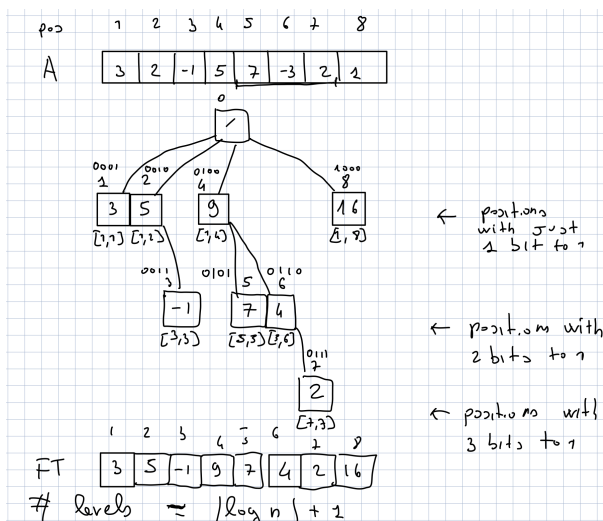
Per iniziare a spiegare questa struttura dati, vediamo una versione del problema rilassata, ovvero cerchiamo di risolvere solamente la sum , e solo per gli indici delle potenze di 2, quindi nell'array che vedremo saranno solo 1, 2, 4 e 8.



Vediamo il fenwick tree associato a questo problema, dove in ogni nodo troviamo al prefix sum che viene richiesta da quella query, infatti il nodo 4, ha il risultato della prefix sum da 1 a 4. In questa maniera la $\text{sum}(i)$ si risolve accedendo semplicemente al nodo i , ovviamente solo per i potenze di 2. Per risolvere la $\text{add}(i, v)$ basta semplicemente aggiornare i nodi che contengono l'index i , infatti se dovessimo fare $\text{add}(3, 10)$ dovremmo aggiungere 10 ai nodi 4 e 8.

Notare: sum richiede $\Theta(1)$ mentre add richiede $\Theta(\log n)$

Il prof per spiegare come si creano più livelli dell'albero nelle note fa un bel casino, mentre in classe spiegò solo la seguente immagine:



In questa maniera, abbiamo un livello per ogni set di numeri con uno specifico numero di uni nella rappresentazione dei bit. Inoltre si vede come la rappresentazione alla fine del fenwick tree non sia altro che un array, di lunghezza uguale all'array (il prof dice $n + 1$ ma non capiscooooo).

Notare come abbiamo la prefix sum solo di alcuni subset

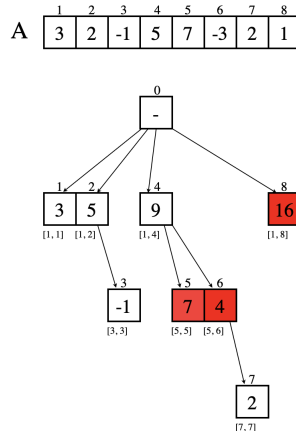
Notare inoltre come è possibile usare solamente il fenwick tree, visto che si possono comunque ricavare i valori precedenti, usando l'operazione $A[i] = \text{sum}(i) - \text{sum}(i - 1)$.

Adesso vediamo come rispondere alle query di Sum e Add .

- $\text{Sum}(i)$: per calcolarla basta andare al nodo i , e da quel nodo andare attraverso i parent al nodo radice, quindi dobbiamo implementare la $\text{parent}(i)$, e il costo è $\Theta(\log n)$

- **Parent(i)**: per calcolarlo si rimuove il **trailing one** del figlio, ovvero l'uno nella rappresentazione binaria più a destra (più piccolo), e questo 1 possiamo isolarlo facendo $k = i \& -i$ (questo perché la rappresentazione in negativo dei binari è in complemento a 2, ovvero negato del numero + 1) e quindi basta fare $i - k$ per avere il genitore di i .

- `Add(i,v)`: dobbiamo risolvere il problema di aggiornare



Un esempio dei nodi da aggiornare se dovessimo fare la `add(5, _)`, vediamo come si devono aggiornare i nodi alla destra della i e i nodi genitori sempre alla sua destra.

Per fare questo possiamo notare come tutti i nodi da aggiornare non sono altro che il nodo corrente, sommato al suo trailing one, quindi come per il genitore l'operazione da fare, è $k = i \& -i$ e $i + k$.

Un esempio di implementazione in rust:

```

1  #[derive(Debug)]
2  pub struct FenwickTree {
3      tree: Vec<i64>,
4  }
5
6  impl FenwickTree {
7      pub fn with_len(n: usize) -> Self {
8          Self {
9              tree: vec![0; n + 1],
10             }
11         }
12
13         pub fn len(&self) -> usize {
14             self.tree.len() - 1
15         }
16
17         /// Indexing is 0-based, even if internally we use 1-based indexing
18         pub fn add(&mut self, i: usize, delta: i64) {
19             let mut i = i + 1;
20             assert!(i < self.tree.len());
21
22             while i < self.tree.len() {
23                 self.tree[i] += delta;
24                 i = Self::next_sibling(i);
25             }
26         }
27
28         /// Indexing is 0-based, even if internally we use 1-based indexing
29         pub fn sum(&self, i: usize) -> i64 {
30             let mut i = i + 1;
31
32             assert!(i < self.tree.len());
33             let mut sum = 0;
34             while i != 0 {
35                 sum += self.tree[i];
36                 i = Self::parent(i);
37             }
38
39             sum
40         }
41
42         pub fn range_sum(&self, l: usize, r: usize) -> i64 {
43             self.sum(r) - if l == 0 { 0 } else { self.sum(l - 1) }
44         }
45
46         fn isolate_trailing_one(i: usize) -> usize {
47             if i == 0 {
48                 0
49             } else {
50                 1 << i.trailing_zeros()
51             }
52         }

```



```

52     }
53
54     fn parent(i: usize) -> usize {
55         i - Self::isolate_trailing_one(i)
56     }
57
58     fn next_sibling(i: usize) -> usize {
59         i + Self::isolate_trailing_one(i)
60     }
61 }

```

8.1 Counting Inversions in an Array

Dato un array $A[1, n]$ di n interi positivi, contare il numero di inversioni di A .

Se $1 \leq i < j \leq n$ e $A[i] > A[j]$ allora si chiama **inversione** di A .

Esempio: $A=321654$ dove $\#inversions=6$, ovvero $(3, 1) (3, 2) (5, 4) (6, 5) (6, 4)$

Target solution: $\Theta(n \log n)$

Se i valori sono $\Omega(n)$, quindi superano di valore la dimensione dell'array, possiamo rimapparli per essere in $[1, n]$, e vogliamo tenerci sempre il loro ordine relativo nell'array così fa fare il discorso del $i < j$, si sortano in ordine crescente, e si sostituisce al valore il suo rank nell'array ordinato .

Per esempio $A = 422010100$ diventa $A_{\text{remap}} = 3214$

```

1 pub fn counting_inversions(a: &[u64]) -> usize {
2     if a.is_empty() {
3         return 0;
4     }
5
6     let max = *a.iter().max().unwrap() as usize;
7     let mut ft = FenwickTree::with_len(max + 1);
8
9     let mut count: usize = 0;
10    for &e in a {
11        count += ft.range_sum((e + 1) as usize, max) as usize;
12        ft.add(e as usize, 1);
13    }
14
15    count
16 }

```

9 100 prisoners

Trump dà un'ultima possibilità di freedom a 100 scenziati, trump dà un numero da 1 a 100 ad ogni scenziato.



C'è una stanza con 100 cassetti, ognuno contiene un numero ma non si conosce, ogni scenziato entra e può aprirne 50. Gli scenziati vincono se tutti trovano il loro numero. si comunicano all'inizio ma dopo non possono.

- Trivial Solution: nessuno si mette d'accordo e ognuno apre a caso. $\frac{1}{2}$ oppure il 50% di successo per il singolo ma $\frac{1}{2^{100}}$ che tutti vincono, coglioniiiiii.

Si usa una strategia adaptive, per aprire i drawer.

1. Scenziato numero i apre il drawer i con numero d_i , se $d_i == i$ allora vince.
2. Se no apre il drawer d_i e continua così.



Ovviamente è un ciclo, quando lo percorriamo tutto si arriva alla soluzione, perché l'ultimo è il nostro, quindi per vincere dobbiamo prendere un ciclo minore di 50, quindi quale è questa probabilità?

IMG MANCANTE
Screenshot_20251110_111744.png

IMG MANCANTE
Screenshot_20251110_112458.png

Si ma non esistono cicli in questi drawer, c'è solo un ciclo, cambia solo da che nodo ci entri.

Quindi l'equazione diventa tutte le permutazioni possibili, l'ordine che hanno visto che lo sappiamo già, e poi gli elementi che mancano:

IMG MANCANTE
Screenshot_20251110_113544.png

quindi come si calcola? si fa:

IMG MANCANTE
Screenshot_20251110_113709.png

Vediamo che non è brutto adesso, rispetto alla soluzione trivial, abbiamo una probabilità più grande yeeee.

10 Super clever algorithm

10.1 Duplicate Eleemnts in an array

Siamo un un array $A[0, \dots, n]$ di $n+1$ numeri in $\{0, \dots, n-1\}$, dobbiamo trovare almeno un elemento duplicato.

Si usa una bit map per vedere se abbiamo già visto quel valore:

IMG MANCANTE
Screenshot_20251110_114419.png

Complessità, $\Theta(n)$ tempo, ma anche $\Theta(n)$ space, quindi dobbiamo sistemarlo. Sembra impossibile perché già usiamo solo 1 bit per risolvere questo problema.

10.1.1 log n passes

Si fanno $\Theta(\log n)$ passes su A, ovvero si ricostruisce l'array e (bitmap) bit per bit.

IMG MANCANTE
Screenshot_20251110_115155.png

si guardano prima le cifre più alte dei numeri, poi si guardano quelle intermedie and so on, così si fa lungo $\log n$ e, e ci settiamo solamente chi troviamo

IMG MANCANTE
Screenshot_20251110_115344.png

Greve in questa maniera ci troviamo i duplicati direttamente alla fine. Per ogni cifra si conta chi ne ha di più, se troviamo più 1 o più zeri ce lo segniamo e si restringe via via il campo. Notiamo che è molto bellino ma solo se facciamo niente random access, se no siamo più lenti di prima insomma.

10.1.2 Destroy A

Ma se potessimo fare random access possiamo andare più veloce? possiamo ma si distrugge A, quindi dobbiamo fare una copia in caso. Questa soluzione deriva dai 100 prisoners.

Si spostano i numeri nella loro posizione corretta, e quindi basta controllare la loro posizione una volta che si trovano (random access), e via.

IMG MANCANTE
Screenshot_20251110_120010.png

10.2 Floyds cycle finding

Trovare l'inizio di un ciclo in una lista in $\Theta(1)$ extra space, la lista è read only

IMG MANCANTE
Screenshot..20251110..151924.png

Si usano 2 puntatori, S Slow si muove solo 1 step, e F Fast che si muove 2 step by 2.

1. Prima si trovano nel ciclo, così da certificare il fatto che il ciclo esiste (lo sappiamo per cui seccederà).
2. Poi riportiamo F all'inizio e lo settiamo con speed 1, mentre intanto S continua.

MAGIA: Quando si ritroveranno, si ritroveranno all'inizio del ciclo. Questo perché S fa m steps, mentre F fa $2m$ steps, si chiama a la lunghezza del prefisso prima dell'inizio del ciclo, mentre b quanto sono dentro al loop, per cui:

IMG MANCANTE
Screenshot..20251110..152526.png

IMG MANCANTE
Screenshot..20251110..152927.png

$$m = a + b$$

$$2m = a + b + k * \text{length of the loop (}=b)$$

$$a = k * l - b$$

10.3 Majority Element

Dato un array $A[1, n]$ trovare se c'è, l'elemento che occorre almeno $\frac{n}{2} + 1$ volte, quindi la maggioranza.

Soluzioni:

- Sort $\Theta(n \log n)$ time $\Theta(n)$ space
- hashmap $\Theta(n)$ * time $\Theta(n)$ space (l'hashmap è probabilistica dipende)

possiamo fare meglio in spazio? vogliamo tempo lineare $\Theta(n)$ e spazio costante $\Theta(1)$,

10.3.1 Boyer Moore Algorithm

Abbiamo un array.

Si prende un candidato c e a un counter, si processa da sinistra a destra, e ad ogni iterazione abbiamo $A[i]$, e se $A[i] == c$ si aumenta il counter di 1, se no :

IMG MANCANTE
Screenshot_20251110_154532.png

IMG MANCANTE
Screenshot_20251110_154632.png

10.3.2 Support Insertion/Deletion

Abbiamo un array, si inserisce nella sequenza, oppure si fa la deletion, e dopo ogni singola operazione ci chiediamo quale è il majority. E vogliamo $\Theta(\log n)$ time e space.

IMG MANCANTE
Screenshot_20251110_155657.png

Si fa la binary representation per questa robetta, e si contano il numero di 0 e di 1.

IMG MANCANTE
Screenshot_20251110_160017.png

bellissimi, adesso se facciamo un sacco di operazioni di delete e una insert, come si fa? basta che si fa la update dei nostri calcoli, una delete

IMG MANCANTE
Screenshot_20251110_160353.png

10.4 Misra-Gries Heavy Hitter

Dato un array $A[1, n]$, trovare l'elemento $T - 1$ che occorre almeno $\frac{n}{T} + 1$ volte, stiamo generalizzando il problema majority, con $T = 2$ è quello.

Si tiene un set K di T candidati e counters,

IMG MANCANTE
Screenshot_20251110_161014.png

Abbiamo $\Theta(T \cdot n)$ Time, godo

11 Alice and bob

Alice seleziona prima e Bob seleziona dopo, $2n$ numeri, Regola di Scelta: Un giocatore può prendere un numero solo da una delle due estremità della fila rimasta. Obiettivo: Ogni giocatore vuole massimizzare la somma totale dei numeri che ha raccolto. Alice vince se la sua somma finale (SA) è maggiore o uguale a quella di Bob (SB).

L'intuizione chiave: Scegliendo un numero da una posizione dispari (la 1°), Alice costringe Bob a scegliere tra due numeri che sono entrambi in posizioni pari. Qualunque numero Bob scelga, "scoprirà" un nuovo numero in posizione dispari per Alice.

Alice può continuare così per tutto il gioco, prendendo sempre un numero dal set "Dispari", e Bob sarà sempre costretto a prendere un numero dal set "Pari".

grazie gemini! insomma lei prima calcola quale è la somma vincente, se quella nei numeri pari o nei numeri dispari (posizioni negli array) e così così gasse.

12 Fenwick Tree binary indexed tree, BIT

12.1 Dynamic Prefix Sums

Dato un array $A[1, n]$ vorremmo supportare le seguenti operazioni: IN LOGARITMIC TIME.

- $\text{Sum}(i) = \sum_{k=1}^i A[k]$
- $\text{Add}(i, v) = A[i] + v$

$$\text{RangeSum}(i, j) = \text{Sum}(j) - \text{Sum}(i - 1)$$

Solution Prendiamo una versione rilassata del problema, la add rimane uguale, ma la sum si fa solo per le posizioni i che sono una potenza di due:



Qui creiamo un albero per la prefix sum, solamente per le potenze di due, così se ci chiedono la sum la sappiamo, e se chiedono la add, possiamo farlo, tipo add 2 in posizione 3, basta che aggiungiamo nel nodo 4 e 8 dell'albero 2, e si fa un tempo logaritmico.



Adesso visto che facciamo questa cosa solo per le potenze di due, vediamo come tutti i nodi hanno solamente un bit settato a 1.



Qui si continua con 2 bit a 1, e vediamo come i figli vengono messi seguendo i bit del padre, il nodo 3 ha 2 bit dopo il nodo 2, uguale il 5 e il 6 hanno 2 bit dove il padre ne ha solo 1, mi so spiegato di merda ma si vede via. Praticamente il prefisso è del padre e il suffisso è del figlio, così si capisce meglio.



Adesso cerchiamo di capire la regola, manca solo il nodo 7, dove va messo? si mette nel livello che ha quanti 1 ci sono nella sua rappresentazione binaria, $7 = 0111$ quindi la terzo livello, e il suo padre chi sarà? 6 ovviamente, il suo prefisso è 011 e quindi suo figlio sarà per forza 0111

Quindi non serve una structure per il tree, non ci serve nemmeno l'array iniziale, si tiene solo il nostro FenwickTree dentro l'array, tanto con sum si ricostruisce tutto, quindi 0 extra space.

Come si fa la sum? si parte dalla posizione chiesta? quindi sum(3) si parte dal nodo tre dell'albero e si somma con i suoi genitori, quindi ci serve l'operazione di parent() e basta per fare la sum, quindi gasse.

IMG MANCANTE
Screenshot_20251114_144043.png

IMG MANCANTE
Screenshot_20251114_144527.png

La cosa figa di questa struttura è che ha 0 extra space e il modo per calcolare il parent super easy. Quale è la sua **altezza**? $\log n$, questo perché sono tutte le potenze di 2. godo

IMG MANCANTE
Screenshot_20251114_145327.png

Per fare la add è la solita storia, dobbiamo andare a destra e in alto, quindi gasse dobbiamo trovare il nodo a destra, ed è come il parent ma invece che togliere il trailing one (appunti vecchi gasse) si aggiunge.

IMG MANCANTE
Screenshot_20251114_150246.png

IMG MANCANTE
Screenshot_20251114_150327.png

12.2 Counting Inversion

Dato $A[1n]$ di numeri positivi $\leq n$ contare il numero di inversions in A .

Una inversion è $i < j$ such that $A[i] > A[j]$.

IMG MANCANTE
Screenshot_20251115_152154.png

Questo conta quanto non è sortato l'array, ovviamente il bound è quando l'array è sortato in revers order.

Soluzione trivial, fare insertion sort o contare gli steps, ma è $\Theta(n^2)$ time. C'è anche una soluzione mergesort based n logge enne ma non ci freca (basta contare le singole inversioni che fa il mergesort).

IMG MANCANTE
Screenshot_20251115_153936.png

FT based: si costruisce il FenwickTree dove in ogni posizione $F[j]$ ha la frequenza dell'elemnto di $A[i] = j$, si fa uno scan da sinistra a destra di A, e si da l'update di questa bellissima cosa. Mentre popoliamo il FT è facilissimo vedere quante inversioni abbiamo all'interno di A, basta fare una rangesum dalla posizione dopo il nostro j fino alla fine del FT, quindi easy. come si vede dall'immagine si arriva a 1 e basta fare la somma di tutto l'array dopo e gasse. Quindi $\Theta(n \log n)$ time.

IMG MANCANTE
Screenshot_20251115_154448.png

In caso il max value fosse troppo grosso, chiaramente non dobbiamo fare un FT enorme, ma basta remappare. Si sortano e diamo il rank, e si sostuiscono i valori con il loro rank, tnato ci importa delle inversion non dei valori effettivi, godo.

IMG MANCANTE
Screenshot_20251115_154532.png

12.3 Nested Segment

IMG MANCANTE
Screenshot_20251115_155008.png

IMG MANCANTE
Screenshot_20251115_161220.png

IMG MANCANTE
Screenshot_20251115_162556.png

Facciamo un bel remap con l'inizio e la fine dei segmenti, successivamente si fa uno scan e si mette 1 nelle posizioni dove i segmenti iniziano (sinistra ovviamente diobe).

Quindi ora si fa la range sum dentro il range di un segmento, e infatti gasse si trova la risposta al nostro problema, ma comunque dobbiamo escludere delle cose, 7 e 9 sono di troppo, ma come si fa?

IMG MANCANTE
Screenshot_20251115_162857.png

Si inizia dall'ultimo segmento, che sicuramente va benissimo e si fa la sua range sum, e poi che si fa? siamo una struttura dinamica, allora si elimina il suo inizio, gasse così si toglie per gli altri, così si tratta sempre il segmento corrente come l'ultimo gasseeeeeeeeeeeeeeeeeeeeeeee.

12.4 Update the Array

Dato un array di soli 0 vogliamo supportare 2 operazioni:

- Access(i) = $A[i]$
- Update(i, j, v) = $\forall k \in [i, j]$ allora $A[k] += v$

IMG MANCANTE
Screenshot_20251115_172803.png

Si tiene V ovvero un FT, dove la posizione i è la prefix sum fino ad i di A , ovvero $\forall i A[i] = \sum_{k=1}^i V[k]$, praticamente la prefix sum di V è l'array A . Così se facciamo $+1$ in una posizione di V , questo si propaga direttamente in ogni posizione a destra, in questa maniera se si fa l'update, basta mettere $+v$ in posizione i e $-v$ in posizione $j + 1$, così da cancellare quella propagazione

IMG MANCANTE
Screenshot_20251115_173156.png

Ogni update si trasla con 2 add sul FT per funzionare, mentre un access è semplicemente una somma

IMG MANCANTE
Screenshot_20251115_173247.png

Una cosa che non avevo capito, il FT non è una scatola nera, fa solamente parent e moveright, serve solo a questo, in questo caso la add non propaga come abbiamo visto inizialmente per fare la dynamic prefix sum madonna bestia laida, quindi semplicemente si fa la add sul singolo nodo e gasse così quando facciamo la sum effettivamente è tutto aggiornato, godo. E' vero che in questo caso V (l'FT) è la prefix sum di A in ogni singolo i , godo, e per questo si fanno due add per le update.

12.5 Dynamic Prefix Syms with range update

Dato $A[1,n]$

- $\text{rangeupdate}(i,j,v)$
- $\text{sum}(i)$
- $\text{access}(i) = \text{sum}(i) - \text{sum}(i-1)$

NOTare, la $\text{sum}(i)$ non è come la $\text{sum}_v(i)$ del problema precedente, vogliamo proprio la sum di i in a quindi gasse.

Il sunto è che se gestiamo la rangeupdate come facciamo di solito, non abbiamo dei risultati corretti, in questo caso il prof dice moltiplichiamo per i la sum del FT, ma è corretto solo se si inizia sempre dall'inizio con la rangeupdate diobestia.

IMG MANCANTE
Screenshot_20251117_104717.png

Dobbiamo usare 2 FT, il primo si usa come prima, stessa strategia, ma dobbiamo avere un secondo FT con le correzioni del primo.

IMG MANCANTE
Screenshot_20251117_110140.png

13 Segment Tree

Stesso problema del FT, sempre la add ma un problema diverso per la range sum è più generale, per esempio range min

Ricordarsi: occupa il doppio dello spazio del FT, visto che FT è implicito



Si fa un binary tree sopra i valori, combinando le coppie delle posizioni, nei padri si mettono le sum dei figli, visto che la nostra operazione è quella (?),

Praticamente il funzionamento è: ci teniamo nei nodi la risposta ai nodi sotto di noi.



Ora ci serve una update e un modo per rispondere al problema. E la update deve fare $\Theta(\log n)$ nodi.

Per la range sum come si fa? in questo caso abbiamo query range 2 e 6, e si fa la compare con i node range, e abbiamo 3 casi,

1. Total Overlap: quando il query range contiene totalmente il node range, quindi la risposta del nodo è richiesta per rispondere alla domanda richiesta. Si prende direttamente il valore e lo usiamo per la risposta
2. Partial Overlap: la risposta che cerchiamo è dentro in un qualche sub tree dentro il nostro nodo. Il nodo non ha la risposta ma solamente un suo descendent ha la risposta, quindi si deve continuare la ricorsione, in quale side? si va in tutte e due.
3. No overlap: nessun nodo ha questa risposta, per cui non si prende. Si lascia stare nessun overlap ci stoppiano.



E' un'algoritmo recursive che parte dal nodo, e ha tutti e 3 i casi, Si vede bene che si arriva a certi nodi dove abbiamo del partial overlap, e semplicemente seguiamo quello che abbiamo gia fatto

IMG MANCANTE
Screenshot_20251117_124650.png

Quindi è linear o logaritmico? è linear nel nostro esempio, per cui come si fa? con questi partial overlaps diventa linear a forza di andare a sinistra e destra. Non ho capito però i discorsi del prof, e il running time è $\log n$ insomma, perche si usa toccano almeno $4 \log n$ nodi.

Ora un'altro problema che questa struttura può risolvere:

IMG MANCANTE
Screenshot_20251117_130309.png

IMG MANCANTE
Screenshot_20251117_130309.png

13.1 BMQ with occurrency

IMG MANCANTE
Screenshot_20251117_165416-1.png

Allora si crea l'abero per fare questo bellissimo algoritmo, se comunque dovessimo fare una update viene lo stesso, vabbè fa cagare sto problema perche lo ha spiegatooooooooooooooooooooo

IMG MANCANTE
Screenshot_20251117_165854.png

13.2 problema piu greve

Array con NON NEGATIVE integer, vogliamo:

- $\text{add}(i,v)$
- $\text{Rangesum}(i,j)$
- $\text{search}(s)$: report the position i of the shortest prefix of A such that $\sum_{j=1}^i A[j] \geq s$



Trivial si cerca con una binary search ma viene una merdina, quindi che si fa, dobbiamo fare la ricerca di 23, si inizia dalla root, si dice ma 8 è troppo piccolo per arrivare a 23, quindi si va direttamente a destra perche si cerca 23-8, allora si cerca per 15 visto che togliamo il figlio di sinistra, allora 19 è maggiore o uguale a 15, allora si cerca lì dentro, 6 è più piccolo di 15, allora si cerca 15-6 e infatti 13 è più grande allora gasse.

Quale è il running time? va in $\Theta \log n$

13.3 problema piu greveeeeeee

Dato A di interi:



Notare che abbiamo $\Theta \log n$ mentre abbiamo $\Theta n \log n$ spazio per colpa dell'hashmap.

13.4 problema piu greveeeeeeeeeeeeeeeeeee



Si tiene un bst in ogni nodo, così da cercare il successor nel range in $\log n$ time, Si computa il successore per tutti i total overlapping node, e si uniscono prendendo il minimo, e query time $\Theta(\log^n n)$, mentre space usage $\Theta(n \log n)$, query time can be improved,

13.5 Triplets

Usa segment tree o FT, per risolvere un problema, dato $A[0, n-1]$ di integer, e vogliamo contare nell'array il numero di tripliete (number of ways of 3 values),

$$i, j, k \quad (0 \leq i < j < k < n)$$

such that

$$A[i] < A[j] < A[k]$$

Esempio:



Si processa l'array da sinistra a destra, e si contano il numero di occorrenze dei valori più piccoli nel prefisso del FT, per 5 tipo c'è solamente un valore prima di 5, ovvero 3, mentre se la query fosse stata 7 allora sarebbe stato 4 la risposta, così si fa il k per ogni elemento nodo, poi si fa questa cosa anche da destra a sinistra e si riconta.

13.6 Range update with Lazy Propagation

Dato $A[1, n]$:

- Rangeupdate(i, j, v)
- Sum(i)

Ricordarsi che se non è sum ma è access possiamo usare FT con la soluzione già vista, e comunque abbiamo visto anche per sum, ma è solo specifica per quello, mentre con ST possiamo fare molte più operazioni.

Soluzione: Si tengono due ST, che tiene l'operazione che vogliamo, quindi in questo caso la somma, ma se ne tiene un'altro il **Lazy Tree** dove ci mettiamo gli update che ancora dobbiamo propagare, ovvero che facciamo, ma ancora non ci sono nell'altro ST.

IMG MANCANTE
Screenshot_20251118_110350.png

Facciamo la $\text{RangeUpdate}(0,4,20)$, ogni volta che abbiamo una partial overlap si fa l'update dentro ST, si parte dalla root, abbiamo partial overlap, (il range è $0,4$), quindi si aggiunge $+100$ alla root, questo perché abbiamo da fare l'update di 5 nodi, $20 * 5 = 100$.

ricordiamo le regole:

- partial: si fa l'update su ST
- Total: si fa solamente l'update sul Lazy, solo se non è una leaf.

IMG MANCANTE
Screenshot_20251118_111127.png

Ora dobbiamo scegliere tra destra e sinistra, a sinistra abbiamo total overlap, e destra abbiamo partial overlap, quindi sul nodo di sinistra si fa il $+20$ nel lazy (in realtà sarebbe 80) ma è implicito perché tanto va fatto anche nei nodi sotto, quindi ci ricordiamo solo il 20 nel lazy (secondo me uno fa confusione proofofofofofofo). Nel nodo di destra come detto abbiamo PO, quindi si fa $+20$, e questo perché la porzione di intervallo è solamente di un nodo, quindi solamente $+20$.

IMG MANCANTE
Screenshot_20251118_122851-1.png

Adesso sotto 21, abbiamo che 19 ha partial overlap, quindi si fa $+20$. Sotto 19 abbiamo 6 e 13, e 6 è un total overlap ma è una leaf quindi si scrive direttamente. A destra niente e anche a destra di quello sopra niente

Quale è il running time? come con sum, si toccano $\log n$ nodi, ma per ogni nodo quanto tempo? partial overlap è constant time, questo perché si toccano tutti i suoi nodi che servono, no overlap constant time, mentre total overlap? si mettono nel lazy tree quindi sempre constant time. Quindi $\Theta(\log n)$ time.

IMG MANCANTE
Screenshot_20251118_123658.png

BELLISSIMOOOOOOOOOOOO

Ora come si fa la Sum? $\text{Sum}(3) = \text{RangeSum}(0,3)$, si guarda per ogni nodo se nel lazy tree c'è un **pending update**, se c'è allora si aggiorna il nodo corrente che ci dà la risposta, e si aggiunge nel lazy tree il pending update dei nodi sotto, così se mai dovessero servirci si facciamo l'update. Ora abbiamo la risposta alla nostra sum e via.

Quindi per un range update non si paga subito il costo ma si paga solo quando serve, così abbiamo tempo logaritmico. Anche la Sum va in $\Theta(\log n)$.

Altri appunti per cercare di capire meglio:

IMG MANCANTE
Screenshot_20251120_120530.png

IMG MANCANTE
Screenshot_20251120_120621.png

13.7 Persistent Segment Tree

Vogliamo che la data structure sia persistent, ovvero abbiamo delle operazioni che fanno l'update della data structure, e certe che fanno le query, adesso però possiamo dare un timestamp con la query e vogliamo lo snapshot della struttura e via.

Dato $A[0, n - 1]$

- Add(i,v)
- RangeSum(i,j,t) dove t è il timestamp

Facciamo piu soluzioni stupide:

- Facciamo una copia del ST dopo ogni update, pro: range sum in $\Theta(\log n)$ time ma cons: space usage $\Theta(n \cdot t)$ dopo t update, inoltre add operation prende $\Theta(n)$ time (perche si copia tutto il ST).
- Exploit: non ha senso copiare visto che non si cambia tutto, quindi abbiamo solamente n $\Theta(\log n)$ nodi dopo ogni update:

IMG MANCANTE
Screenshot_20251118_154518.png

Si fa questa bella add, e vediamo che cambiamo questi nodi, allora per ogni nodo si tiene tutti i valori che hanno preso durante il tempo, quindi non ci teniamo solo un val-

ore ma una lista di valori e di timestamp, in questo caso vediamo che 21 ha solo 21,0 visto che non è stato cambiato.

Adesso facciamo un'altra add, e vediamo che accanto 21 viene 41,2, visto che godo.



Pro: add prende $\Theta(\log n)$ e space $\Theta(n + t \cdot \log n)$ e range update prende

RangeSum(i,j,t) cerca per il predecessore di t in ogni lista dei Total Overlapping Node, Time Complexity: $\Theta(\log n \cdot \log t)$ ($\log t$ perche si fa una binary search dentro la lista)

Finalmente soluzione FIGA: Exploitiemo il fatto che tutti i $\log n$ nodi sono da una root-to-leaf path. Aggiungiamo un array. Si replica i nodi che cambiano e basta insomma.



in questo caso il nodo in posizione 3, ovvero 4 con 14, e dobbiamo cambiare anche il suo genitore, e si fanno proprio delle copie e si mettono al posto di quelli vecchi, ma comunque si utilizza l'array per puntare alla nuova root.

quindi è tutto come se fosse solamente un ST, semplicemente a seconda il timestamp cambia solamente lo starting point.



Se facciamo un'altra add semplicemente ci prendiamo i nodi sempre del timestamp diverso, e come si fa? il parent deve essere sempre the last one, quindi è easy. tanto per fare una ricerca si parte dalla root e quella si prende sempre con l'array.

- RangeSum prende $\Theta(\log n)$ time
- add prende $\Theta(\log n)$ time
- space usage prende $\Theta(n + t \log n)$ dopo t update

14 Mo's Algorithm

D'ora in poi si prenderanno un sacco di queryrange (ancora), Se è una sum query con update si può risolvere con un FT, mentre per le altre si possono risolvere con ST, ma deve essere possibile suddividere la query in sub query ed è così che si usa l'ST. MA NON PER TUTTEEEE

14.1 Problem meno difficult

Dato $A[1, n]$ di colori (valori $\leq n$), e un set di Q di q queries (OFFLINE).

- 3 o più (i,j): report # of distinct color that occur at least 3 times in $A[i, \dots, j]$

Target time complexity: $\Theta((n + q)\sqrt{n})$ time

Facciamo una **trivial solution**: $\Theta(n \cdot q)$ time



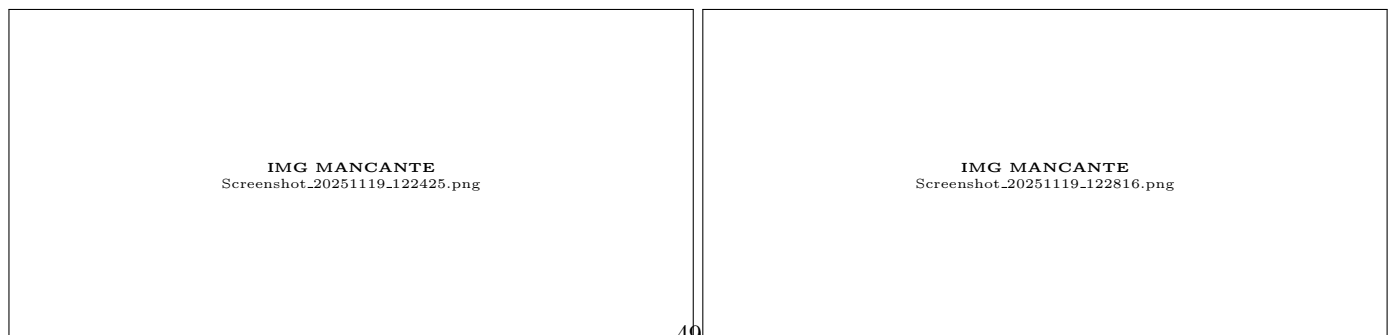
Si usa un array di counters per contare il numero di occurences

Adesso vorremmo usare la risposta alla prima query per dare una mano alla seconda, abbiamo un array di counters e abbiamo la variabile **Answer** che ci conta il numero di colori con almeno 3 occurences.



Abbiamo la seguente query, e sappiamo la sua risposta, se la prossima sarà un range diverso da quello della prima, si toglie semplicemente la parte che non ci interessa e via, ovviamente se non c'è overlap stiga

Soluzioe definitiva per questo tipo di trick:



Notare che abbiamo sempre: $\Theta(n \cdot q)$ time però l'ordine delle query serve, quindi che strategia si usa? vogliamo l'ordering per minimizzare il numero di elementi da togliere e da aggiungere, ma ancora non andrebbe a ridurre il running time.

Notare: esiste un ordering delle queries che permette un time complexity di $\Theta((n + q)\sqrt{n})$.



Si raggruppa le query dal loro left extreme (**cur_i**), in gruppi di size $\sqrt{n}$. (si fa il sort (i,j) by $(i/\sqrt{n}, j)$)

In ogni gruppo si fa il sort del right extreme (**cur_j**), e si vanno a risolvere le queries.

Siamo in un gruppo abbiamo **cur_i** e **curr_j**, consideriamo **cur_j**, che alle brutte dentro il gruppo può arrivare fino a n. Quindi per ogni gruppo al massimo facciamo n "move", ovvero dobbiamo fare l'add n volte.

quindi overall $\Theta(n\sqrt{n})$ moves (o calls to add).



Invece quante moves facciamo in base a **cur_i**? siamo nella situazione dell'immagine, ovvero si distano al massimo di $\sqrt{n}$, quindi se nel gruppo ci sono b_i queries, allora si fanno al massimo $\Theta(b_i \cdot \sqrt{n})$ moves per gruppo i

Quindi quante in tutto? $\Theta(\sum_{i=1}^{\sqrt{n}} b_i \cdot \sqrt{n}) = \Theta(\sqrt{q})$

Quindi in tutto $\Theta(n \cdot \sqrt{n} + q \cdot \sqrt{n}) = \Theta((n + q)\sqrt{n})$

14.2 A Difficult Problem

Dato un array $A[0, n-1]$,

- Answer q queries (OFFLINE)

Online ovvero fare q queries una dopo l'altra e una risposta dopo l'altra, mentre OFFLINE abbiamo tutte le query in una volta e si deve dare la risposta a tutte le query insieme.

- $\text{Query}(l, r) = \sum_{e \in D(l, r)} e \cdot K e^2$

$D(l, r)$ = set of distinct element in $A[l \dots r]$,

$K e$ = numero di occurences di e dentro $A[l \dots r]$

quindi per ogni distinct element nel range dobbiamo computare quel lavoro, $K e$ è il numero di occurence dei distinct element

IMG MANCANTE
Screenshot_20251118_184037.png

Target time complexity: $\Theta((n + q)\sqrt{n})$ time

Ora discutiamo della versione statica del problema.

(ORA SAPPIAMO COME FARE LA PARTE SOPRA ERA PRIMA GASSE)

Si fa come prima, ma semplicemente cambia la contribuzione all'interno di answer:

IMG MANCANTE
Screenshot_20251119_153625.png

IMG MANCANTE
Screenshot_20251119_153701.png

15 (STATIC) RMQ

Abbiamo fatto prefix sum static, che si risolve con un array, prefix sum dynamic che si risolve con FT, altre range queries che si risolvono con un ST, inoltre abbiamo fatto anche cose piu carine offline con mos algorithm.

Range Minimum Queries, sono piu difficili di prefix sum, nel caso **dinamico** si possono risolvere con ST con tempo logaritmico, ma nel caso statico si possono risolvere in maniera lineare.

Dato $A[0, n - 1]$

- $\text{RMQ}(i, j) = \text{argmin}(A[i \dots j])$

IMG MANCANTE
Screenshot_20251119_155819.png

Target $\Theta(1)$ time e $\Theta(n)$ space (ST fa $\log n$ per query)

Prima soluzione brutta: $\Theta(1)$ time e $\Theta(n^2)$ space, si fa una tabella con tutte le possibili risposte e via gasse.

Seconda soluzione brutta: sparse array, $\Theta(1)$ time e $\Theta(n \log n)$ space, si fa l'exploit del fatto che due range che si sovrappongono hanno la risposta di una query che li

IMG MANCANTE
Screenshot_20251119_160356.png

IMG MANCANTE
Screenshot_20251119_164341.png

Facciamo una tabella piu piccola, si tengono le risposte per $[i, i + 2^k]$ per ogni k , per la query si fa:

IMG MANCANTE
Screenshot_20251119_164722.png

Soluzione 3: $\Theta(\log n)$ time e $\Theta(n)$ space



Creiamo dei blocchi dentro A lunghi $\log n$ e poi si tiene A' dove teniamo l'elemento minimo di ogni blocco, ovviamente A' è grande $\frac{n}{\log n}$ e si usa la soluzione di prima



questo è il modo per risolvere le query, così abbiamo tempo costante per il total overlap, mentre per calcolare il prefix (verde) e il suffix (rosso) della query che non sono direttamente su un blocco di A' si fa semplicemente uno scan e questo costa al massimo $\log n$ e infatti si godeeeeeeeeeee

ULTIMA SOLUZIONE: $\Theta(1)$ time e $\Theta(n)$ space Solo l'intuizione

15.0.1 Cartesian Tree

2 regole:

- Ogni nodo ha sotto sempre uguale o maggiore,
- La posizione dei nodi nell'albero deve rispettare l'ordine originale dell'array.

Se ci sono duplicati, allora si prende quello più a sinistra.

IDEA HO CAPITO, si prende sempre il minimo e da lì si splitta in due, e così via si prende sempre il minimo da sinistra, godooooooooo



Si crea l'albero on top di A, ci mettiamo dentro il minimo ovviamente, la root è l'elemento minimo, si splitta l'array in due parti.

Il valore più piccolo è 1, per cui è la nostra root ovviamente quello più a sinistra, allora dopo c'è 2, e sempre uno a destra come figlio, successivamente il piccolo a destra è sempre 1 e così via fino in fondo.



Quindi in questo caso $RMQ(4,8) = 1$, perché la risposta alla query è sempre la root di quel sub tree, ovvero $LCA(4,8)$ lower common ancestor



Quindi basta supportare LCA per rispondere alla query, ma noi abbiamo dei blocchi piccoli, si usa le balance parenthesis, quando si trova un nodo per la prima volta si mette

un parantesi aperti, quando si smette di visitarlo, quindi non ci passiamo piu, ne metteremo una chiusa, in tutto sono **2n bits**



In questa maniera possiamo ricostruire tutto il tree, GODOOOOO, la prima parentesi è la root, e così via si ricostruisce, ma quindi servono **4n bit** perché dobbiamo anche capire se il figlio è a destra o a sinistra.

recap: come si fa questa cosa? si rappresenta ogni blocco dentro un cartesian tree e si trova il LCA con questi bit strani.

ROBA TEORICA NON TRIVIAL COME DICE IL PROF: Calcoliamo il tempo, mettiamo che stavolta si fanno i blocchi di $\frac{1}{8} \log n$ bits quindi il cartesian tree si rappresenta con $4 \cdot \frac{1}{8} \log n = \frac{1}{2} \log n$ bits. Quindi abbiamo $2^{\frac{1}{2} \log n}$ possible tree representation ma questo è $2^{\frac{1}{2} \log n} = \sqrt{n}$.

Quindi insomma si fanno tutti i possibili tree (? TODO: chiedere) e abbiamo sempre la risposta in constant time.

15.1 Colored Range Query Problem

Dato $A[1, n]$ di colori (integer $\leq n$)

- Distinct (i, j) ritorna all distinct color in $A[i, j]$ in tempo proporzionale a $k = \#$ di distinct color in $A[i, j]$

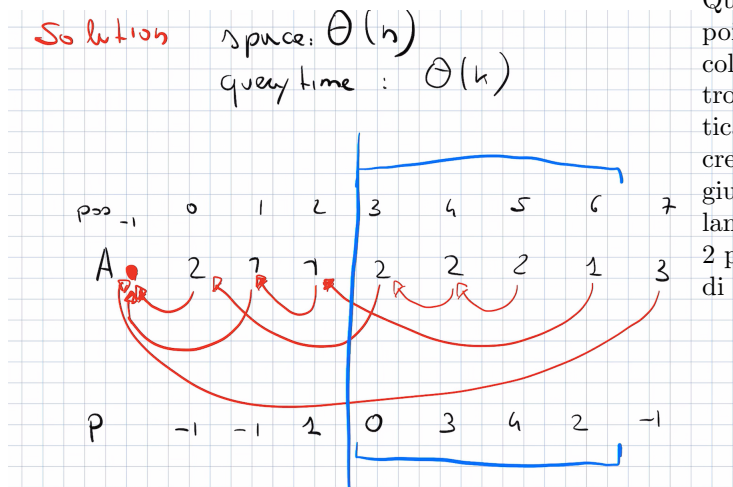
IMG MANCANTE
Screenshot_20251119_193623.png

Prima soluzione: Segment tree con hashset, $\Theta(k \cdot \log n)$ e space $\Theta(n \log n)$

Bella soluzione: Space $\Theta(n)$ e query time $\Theta(k)$ quindi ottimaliiii

IMG MANCANTE
Screenshot_20251119_195246.png

Abbiamo il nostro array, e per ogni valore: se è la prima occorrenza si connette al "PUNTO ROSSO" e se invece si trova una occorrenza si connette alla occorrenza prima. Poi ci teniamo un array dei pointer semplicemente, così sappiamo a chi si chiede. Abbiamo il nostro query range che va a 3 a 6, e vediamo che ogni distinct element ha **esattamente una freccia che crossa la linea blu**, grazie punto rosso.

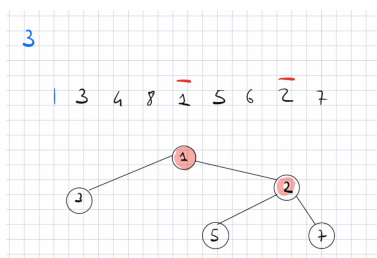


Quindi sapendo questa cosa, basta usare l'array P di pointer e cerchiamo in maniera ricorsiva i numeri più piccoli del min della range query. E ci si ferma quando si trova numeri più grossi di 3, perché tanto non freca, particolarmente si fanno RMQ nell'array P e ricorsivamente si crea il cartesian tree dei range, e non si va ricorsivamente giù, ma si va ricorsivamente a destra e a sinistra, e si fa solamente la ricorsione per elementi minori di 3, se ne perde 2 più o meno per ognuno quindi siamo sempre nell'ordine di k .

Bella merda ho scritto ma si capisce dai su.

15.2 Range Minimum Querys

Abbiamo un array e si chiede di reportare gli elementi più piccoli di un certo k :



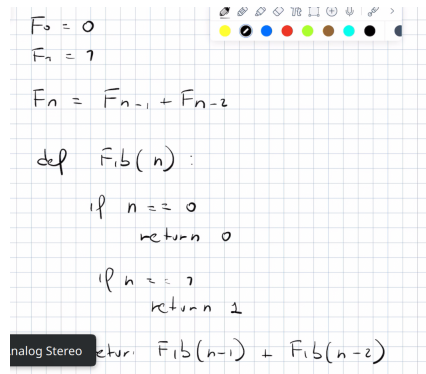
Abbiamo l'array e sicuro dobbiamo riportare 1 e 2, si inizia e si chiede il minimo di tutto l'array, allora si fa una ricorsione a sinistra e destra e si continua così a chiedere il minimo, e ci si ferma ovviamente quando il minimo è più grande o uguale a k .

Quanto costa? si sta costruendo un cartesian tree quindi comunque $O(k)$

16 Dynamic Programming

16.1 Fibonacci Numbers

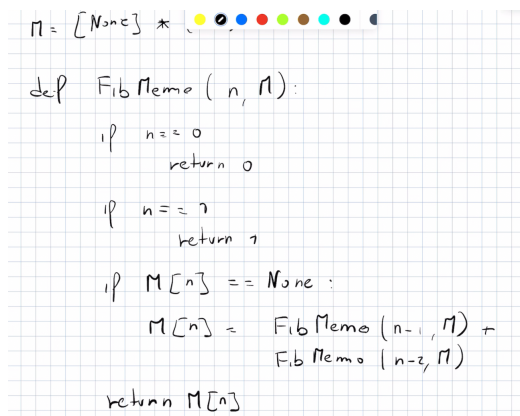
Riusiamo tutto quello che abbiamo già fatto godendo nel mentre.



```
F0 = 0
F1 = 1
Fn = Fn-1 + Fn-2

def Fib(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return Fib(n-1) + Fib(n-2)
```

Si fa schifo questo algoritmo, ci teniamo le risposte in una tabella e si risponde continuamente.



```
M = [None] * 1000

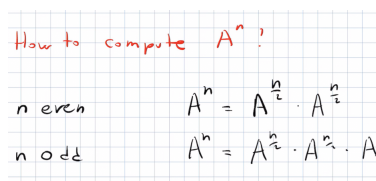
def FibMemo(n, M):
    if n == 0:
        return 0
    if n == 1:
        return 1
    if M[n] == None:
        M[n] = FibMemo(n-1, M) + FibMemo(n-2, M)
    return M[n]
```

TOP DOWN APPROACH: Ci teniamo la tabella M inizializzata a 0.

Possiamo anche avere un bottom down approach cercando di fare un algoritmo più veloce:

16.2 Matrix Exponentiation

Dobbiamo fare A^n , si fa divide and conquer, sappiamo che:

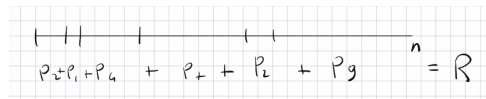


How to compute A^n !

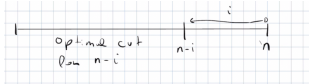
n even	$A^n = A^{\frac{n}{2}} \cdot A^{\frac{n}{2}}$
n odd	$A^n = A^{\frac{n}{2}} \cdot A^{\frac{n}{2}} \cdot A$

16.3 Rod Cutting

Abbiamo una barra di acciaio, di lunghezza n per un taglio di lunghezza $i \in [1, n]$ si prende P_i euro, e vogliamo massimizzare il guadagno, ovvero trovare i cuts che massimizzano la revenue totale.



Soluzione: quali sono i sub problems? solamente smaller input, quindi qui è : Optimal way to cut a rod of length K ; n .



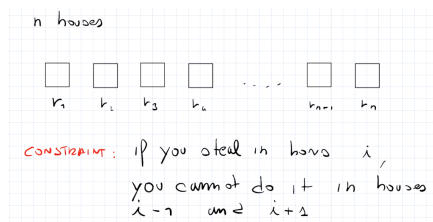
Combine: $R(n) = \max_{1 \leq i < n} \{R(n-1) + P_i\}$ ovvero facciamo il sub problem e andiamo avanti restringendo i .

```
def rod(n):
    R = [0] * (n+1)
    for i in range(1, n+1):
        for l in range(1, i):
            R[i] = max(R[i], R[i-l] + P[l])
    return R[n]
```

$\Theta(n^2)$ time

Prima si inizializza la tabella dei total revenue, poi per ogni i a $n+1$ e per ogni l in che va fino ad i , si prende il massimo tra quello che c'è ora e toglie tutti gli i . $R[i-l] + P[l]$ ovvero tutti i possibili tagli da fare in quel pezzo di i .

16.4 facebook: Robber

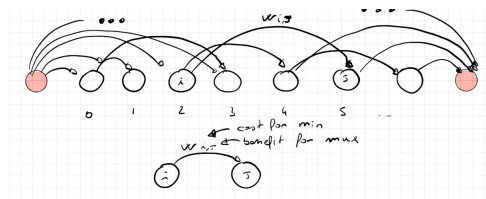


Sei in strada, e ci sono n case, possiamo rubare la casa e prendere la revenue r_n , ma c'è un constraint, se si ruba la casa i non si può rubare in $i-1$ e $i+1$.

Ci serve un tool:

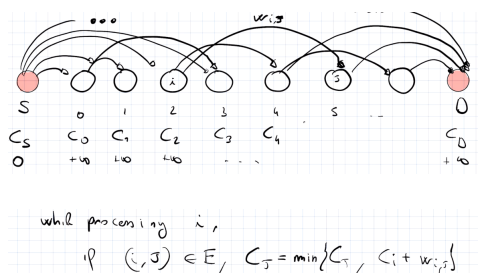
16.4.1 shortest/longest path computation on a DAG

Si definiscono dei nodi speciali, **Source** e **Destination**, si attacca a tutti la source e arrivano tutti alla destination, abbiamo la weight $w_{i,j}$ (il costo) e quindi vogliamo trovare il path dalla source alla destination con il massimo dei pesi oppure il minimo non importa.



Si usa l'algoritmo di dijkistra, abbiamo un heap e ci si mette U nodi dentro con il loro costo, e si estrae quello con costo minore, e si rilassano tutti gli edges che escono da quello, non ci ho capito una sega. costo numero di nodi + numero di archi per log n visto che si parla di heap.

Ma c'è un'altro modo, senza heap, perché se si processano i nodi fino ad arrivare al nodo i in ordine, gli altri non si cagano, questo perché in un DAG tutti gli archi che arrivano ad i si sono già processati, mentre quelli a destra di i ancora non si sono processati, allora cosa si fa?



Si tiene il costo dello shortest path per arrivare a quel nodo c_i , inizialmente sono tutti ∞ , e step by step si computa lo shortest path da sinistra a destra. Per ogni vertex che si processa, si rilassano tutti i vertici.

Ovvero se i, j è un edge, allora il costo per andare a j , diventa il minimo tra il costo di c_j e $c_i + w_{i,j}$, e da sinistra a destra si fa questo calcolo e visto che le weight sono sempre positive, gasse. Traduzione: "Al nodo j conviene tenersi il costo che aveva prima, o arrivarci passando da me (i)?"

16.4.2 Road cutting: ma con il DAG

Si fa il dag con un nodo per ogni cut, e tutti i nodi sono connessi a tutti quelli dopo, e come revenue il guadagno da 1 a i è P_{i-1}

Di nuovo numero di edge più numero di vertex, quindi sempre quadratico in questo caso brutto.

Esempio da gemini:

Il problema consiste nel massimizzare il profitto tagliando un'asta di lunghezza $N = 4$. Modelliamo il problema come la ricerca del cammino a costo massimo (Longest Path) su un DAG, dove i nodi rappresentano i punti di taglio ($0 \rightarrow 4$) e gli archi rappresentano i pezzi venduti.

Dati di Input I prezzi per i tagli sono:

- Taglio da 1m: 1€
- Taglio da 2m: 5€ (Pezzo pregiato)
- Taglio da 3m: 8€
- Taglio da 4m: 9€

Logica dell'Algoritmo Poiché il grafo è un DAG, processiamo i nodi linearmente da sinistra a destra (da 0 a 4). Utilizziamo la formula di rilassamento per massimizzare il profitto:

$$DP[j] = \max(DP[j], DP[i] + P_{(j-i)})$$

In pratica, per ogni nodo, controlliamo se conviene arrivare lì estendendo un percorso precedente.

Esecuzione Step-by-Step

1. Inizializzazione al Nodo 0 (Source)

L'array dei profitti parte vuoto (o a $-\infty$), eccetto la sorgente. Dal nodo 0 lanciamo gli archi verso tutti i nodi futuri:

- Verso 1: $0 + 1 = 1$
- Verso 2: $0 + 5 = 5$
- Verso 3: $0 + 8 = 8$
- Verso 4: $0 + 9 = 9$

Stato Array: [0, 1, 5, 8, 9]

2. Processiamo il Nodo 1 (Profitto attuale: 1)

Proviamo ad estendere il percorso partendo dal nodo 1:

- Verso 2: $1 + 1 = 2$. (Minore di 5, ignoro)
- Verso 3: $1 + 5 = 6$. (Minore di 8, ignoro)
- Verso 4: $1 + 8 = 9$. (Uguale a 9, ignoro)

Nessun miglioramento.

3. Processiamo il Nodo 2 (Profitto attuale: 5)

Qui avviene il rilassamento cruciale (il sorpasso):

- Verso 3: $5 + 1 = 6$. (Minore di 8, ignoro)
- Verso 4: $5 + 5 = 10$. (Maggiore di 9! **Aggiorno**)

Stato Array: [0, 1, 5, 8, 10]

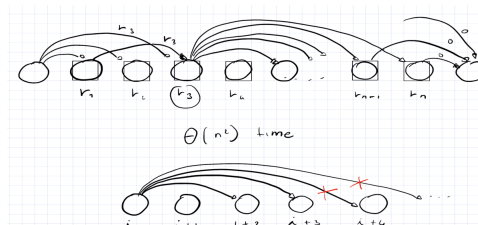
Conclusioni

Arrivati alla fine, il profitto massimo accumulato al nodo 4 è **10€**.

Il percorso vincente sul DAG è: $0 \xrightarrow{+5} 2 \xrightarrow{+5} 4$, che corrisponde a due tagli da 2 metri ciascuno.

16.4.3 Torniamo a facebooooooooook

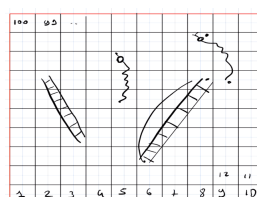
Come si fa con il dag? Se abbiamo un arco che arriva vuol dire che si deve rubare, ma non si può andare nella casa dopo. sempre $\Theta(n^2)$ time



Consideriamo anche che possiamo togliere degli edge, questo perché non ha senso andare da i a $i+4$, perché possiamo prima andare a $i+2$ e poi si va a $i+4$ in questa maniera abbiamo solamente 2 archi che escono da ogni nodo e quindi $\Theta(n)$

16.5 Snake and ladders

Abbiamo una matrice, si inizia dalla cella 1 e dobbiamo raggiungere 100, ci si muove con un dado, e ci sono snake che ti riportano indietro e ladder che ti portano avanti, trovare il minimo numero di tirate di dado per arrivare in fondo.

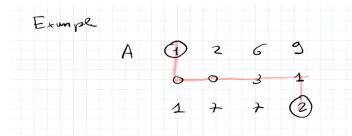


Si trasforma ogni cella in un nodo, e per ogni nodo abbiamo 6 archi (le facce del dado), e se si va su un ladder si va al suo nodo e via, e si computa lo shortest path, e via.

Running Time: $\Theta(n)$

16.6 Minimum Cost Path

A è una $n \times m$ matrice con degli integer, l'obiettivo è computare il path di costo minimo dal top-left angolo al bottom right muovendoci solamente in basso e a destra.



16.7 Longest Common Subsequence LCS

Dato S_1 ($|S_1| = n$) e S_2 ($|S_2| = m$) trovare $LCS(S_1, S_2)$,

Given S_1 ($|S_1| = n$) and S_2 ($|S_2| = m$)
 find $LCS(S_1, S_2)$

S_1 : A B C B D A B $n = 7$
 S_2 : B D C A B A $m = 6$

Vogliamo $\Theta(n \cdot m)$ time.

$LCS(i, j)$ to denote the length of LCS between
 $S_1[1..i]$ and $S_2[1..j]$

Dobbiamo trovare il sub problem ovvero prendere il i prefissi delle due stringhe. E dobbiamo trovare come combinarli, ovvero aggiungere due simboli alle stringhe già processati, ovvero $c_1 == c_2$, se sono uguali gasse, se no stica.

Combine

$LCS(i, j)$

Case #1
 if $c_1 == c_2$
 $LCS(i, j) = 1 + LCS(i-1, j-1)$

Case #2
 if $c_1 \neq c_2$ max {
 $LCS(i, j) = LCS(i, j-1),$
 $LCS(i-1, j)$ }
 ~~$LCS(i-1, j-1)$~~

Recurrence

$$LCS(i, j) = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ 1 + LCS(i-1, j-1) & \text{if } S[i] == S[j] \\ \max \{ LCS(i, j-1), LCS(i-1, j) \} & \text{otherwise} \end{cases}$$

se sono lunghe 0 allora 0, se sonouguali allora gasse, e per gli altri casi uguali.

Ma questo approccio è top down, dobbiamo fare la ricorsione. mentre vorremmo buildare la tabella dall'inizio e piinare la tabella per riga e non per colonna.

Bottom-up Approach

		0	1	2	3	4	5	6
		∅	B	C	A	B	A	
0	∅	0	0	0	0	0	0	0
1	A	0						
2	B	0						
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

$$LCS(i, j) = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ i + LCS(i-1, j-1) & \text{if } S[i] = T[j] \\ \max \{ LCS(i, j-1), LCS(i-1, j) \} & \text{otherwise} \end{cases}$$

Si fa una tabella 2D, inizialmente sappiamo tutte le risposte per la prima riga e colonna, quindi 0, e si usa la recurrence per iniziare a fare la tabella.

Bottom-up Approach

		0	1	2	3	4	5	6
		∅	B	C	A	B	A	
0	∅	0	0	0	0	0	0	0
1	A	0	0	0	1	1	1	1
2	B	0						
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

$$LCS(i, j) = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ i + LCS(i-1, j-1) & \text{if } S[i] = T[j] \\ \max \{ LCS(i, j-1), LCS(i-1, j) \} & \text{otherwise} \end{cases}$$

Notare che si usano le celle in alto, a sinistra e in alto a sinistra per fare calcolare la cella corrente.

16.8 0/1 knapsack problem

0 si prende l'item e 1 no, e si deve prendere più item possibili per massimizzare il peso dello zaino. Abbiamo n items e abbiamo C capacity, ogni item i ha un valore v_i e una weight w_i , prendere un subset di item che massimizza il valore rimanendo nella C capacity.

Example

weight	value	$C = 7$
1	1	$1 + 7 = 8$
3	4	$4 + 5 = 9$
4	5	
5	7	

Vogliamo $\Theta(n \cdot C)$ time.

Cerchiamo il problema per una capacity più piccola e meno item (ovvero si fa un ordering e si prende il prefix via via), e il combine è il seguente:

Combine $K(i, J)$ is an optimal selection for prefix up to i th item with capacity $J \leq C$

Do we want to include item i ?

$$K(i, J) = \begin{cases} K(i-1, J-w_i) + v_i & \text{yes} \\ K(i-1, J) & \text{no} \end{cases}$$

Abbiamo i item mentre prima ne avevamo $i-1$ item, e si vuole prendere l'item i ? No, ovvero si prende la soluzione prima, ma se si prende allora si prende la soluzione di prima + il suo valore, ma dobbiamo fargli spazio, quindi $J - w_i$.

Recurrence

$$K(i, J) = \begin{cases} 0 & \text{if } i=0 \text{ or } J=0 \\ \max \begin{cases} v_i + K(i-1, J-w_i) \\ K(i-1, J) \end{cases} & \text{otherwise} \end{cases}$$

$K(i, J)$ = il valore massimo che posso ottenere considerando i primi i oggetti, avendo uno zaino di capacità J . E si dice che è il massimo con K di $i-1$ con lo stesso spazio, oppure devo aggiungere il mio valore ma devo avere spazio sufficiente, quindi **guardo alla migliore risposta senza il mio peso, per questo $K(i-1, J-w_i)$**

Così è facilissimo, si fa row wise, quindi bottom up, seguendo l'algoritmo gasse,

$K[i, j]$

w	v	i	0	1	2	3	4	5	6	7
1	0	0	0	0	0	0	0	0	0	0
1	1	1	0	1	1	1	1	1	1	1
3	4	2	0	1	2	4	5	5	5	5
4	5	3	0	1	2	4	5	6	6	6
5	7	4	0	2	2	4	5	7	7	7

per ogni cella si prende sempre il massimo valore possibile, e in caso si controlla se togliere l'ultimo item ha senso.

16.9 Fractional knapsack

You can select fraction of items
 $0 \leq p_i \leq 1$ costs $w_i \cdot p_i$
 and value $v_i \cdot p_i$

Si fa in $\Theta(n \log n)$ time, con un algoritmo greedy, per ogni item si fa il $ratio_i = \frac{v_i}{w_i}$ value per unit of weight, si sortano gli item b decreasing ratio, così si massimizza il valore e si selezionano gli item fino a che non si finisce la capacity.

16.10 Puzzle exercise

Abbiamo 2 pairs of coins, selezionare k coins per massimizzare il total value, ma si può selezionare il secondo coin solo se anche il primo è stato selezionato.

n pairs coins

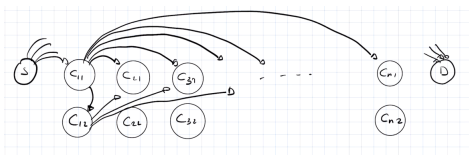
$C_{11}, C_{12}, C_{31}, \dots, C_{n1}$
 $C_{12}, C_{22}, C_{32}, \dots, C_{n2}$

Select k coins to maximize the total value, but you can select the second coin of a pair only if also the first one is selected.

	$\textcircled{10}$	1	$\textcircled{3}$
	1	12	3
$k=1$	10		
$k=2$	10 + 3 = 13		

Con $k=1$ si prende 10 perché possiamo solo selezionare questo, 12 non si può prendere perché si deve prendere 1 prima.

Comunque con $k=3$ non si estende le soluzioni già fatte, perché si prende 10 poi 1 e poi 12, quindi come si fa????



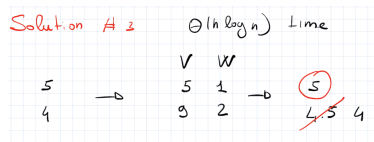
Prima soluzione $\Theta(n^2)$, dag strategy e via, fake source e destination, e si connettono, ma dobbiamo fare lo shortest path di length k .

Solution #2 $\Theta(nk)$ time

V	W
C_{12}	1
$C_{12} + C_{12}$	2
C_{22}	1
$C_{22} + C_{12}$	2
$\vdots$	

it runs in $\Theta(nk)$ time

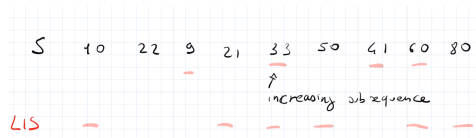
Seconda soluzione, fa $\Theta(nk)$ time, si fa un approach come il knapsack fractional, un coin ha weight 1, un coin e il suo pair ha weight 2, ma non ho capito.



terza soluzione: $\Theta(n \log n)$ si fa un greedy approach, e si usano gli item della seconda soluzione, si fa il ratio e si sortano ma dobbiamo stare attenti di non prendere il coin al primo livello.

16.11 Longest Increasing Subsequence (LIS)

Dato una sequenza S di n elementi, trovare LIS di S. Ovvero una sequenza di elementi che è increasing, ma vogliamo quella più lunga



Prima soluzione quadratica: si prende il sub problem, ovvero si prende solo il prefisso della sequenza, ma qui non serve perché dobbiamo ricordarci i numeri che compongono la LIS, quindi come si fa?

Si forza che LIS(i) finisce con S[i] dove è l'ultimo elemento della LIS, così possiamo fare:

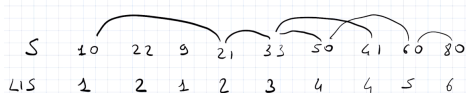
LIS(i) = is the LIS of S[0...i]
which ends with S[i]

Recurrence

$$LIS(i) = 1 + \max \left\{ LIS(j) \mid 1 \leq j \leq i-1 \text{ AND } S[j] < S[i] \right\}$$

Recurrence

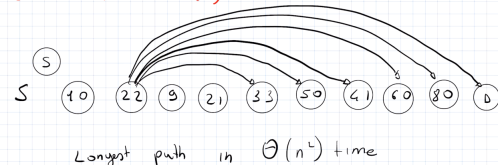
$$LIS(i) = 1 + \max \left\{ LIS(j) \mid 1 \leq j \leq i-1 \text{ AND } S[j] < S[i] \right\}$$



La cosa brutta è che dobbiamo fare il check di tutte le LIS trovate finora, quindi gasse, quadratico.

Solution #2

$\Theta(n^2)$ Time



Soluzione 2: sempre quadratica, si trasforma tutto in un nodo, si fa la fake destination e source, si connette ad ogni nodo i suoi nodi che possono estenderlo nella LIS, e si fa il longest path.

we say that a position i dominates a position j

if $S[i] < S[j]$ AND $LIS[i] \geq LIS[j]$

	i	j
S	10	15
LIS	2	2

3 soluzione $\Theta(n \log n)$, diciamo che una posizione i domina una posizione j se $S[i] < S[j]$ e $LIS[i] \geq LIS[j]$.

Se vale questo j è inutile per i , in questo caso per 16 j è inutile ed è meglio comunque prendere i perché 13 può prendere

Tutti i valori inoltre nei LIS sono rappresentati. C'è solamente una posizione dominante per ogni valore possibile di LIS.

we say that a position i dominates a position j

if $S[i] < S[j]$ AND $LIS[i] \geq LIS[j]$

	i	j
S	10	15
LIS	2	2

Si tiene un valore di Dominating Position, come si piena questa tabella? si cerca un valore più piccolo di quello corrente, abbiamo 10, estende qualcosa? no quindi 1, è 10 una posizione dominante? si va nella tabella e si guarda se c'è

Ora 22 si può estendere, e quindi anche 22 può dominare, MA IL PROF HA INTERROTTO LA LEZIONE

17 Greedy

17.1 Activity Selection Problem

n Attività rappresentate come intervalli, select the largest number of non-overlapping activities

Disegno prof intervalli

Come si affronta, si cerca l'euristica greedy per fare la scelta giusta, ma deve darci la soluzione ottimale, e per fare questa cosa basta trovare un proof oppure un controesempio (E SI TROVA SEMPRE UN CONTROESEMPIO), in questo caso:

1 strategy: si seleziona in base a chi ha lo starting time più piccolo. Si seleziona una attività iniziale, si levano le overlap e si prende quella che inizia prima, va bene? Ovviamente con un esempio stupido non funziona questo esempio.

2 strategy: Select Smallest first, anche questa fa cagare

3 strategy: Selezionare prima le attività che hanno meno overlaps, manco questa è ottimale.

4 strategy: Ending Time, si prende quella che finisce prima e si prende, poi tra quelle che finiscono prima che non overlapano si prendono, questa funziona $\Theta(n \log n)$ time.

Adesso dobbiamo provarlo che è quello ottimale:

Lemma : "there exists at least an optimal solution which includes the first ending activity f "

Proof : dobbiamo provare questo lemma, immaginiamo di aver una optimal solution, chiamata O^*

FOTO PROOOOF

Vogliamo modificare una optimal solution facendo vedere che se la modifichiamo con la nostra strategia allora abbiamo lo stesso risultato.

Diciamo che la sua prima attività g , non è la prima che

finisce, ma ce ne è una f che finisce prima, e se comunque prendiamo questa allora O_p^* rimane comunque optimal.

17.2 Job Sequencing

Per ogni job j , id, deadline e profit, si deve fare il job prima della deadline, e ogni job consuma uno slot di tempo.

FOTO PROF

Quale strategia? si sortano per valore, e poi si processano dal più alto di valore al più basso, dove si mette a? si mette nella sua deadline incredibile, Si buttano gli empty slot in un BST, si prendono in nodi in decreasing order, e ci chiediamo la sua deadline è presa? no? allora ci si mette, se no? si cerca un predecessor della deadline nel BST e così via.

17.3 Magic Numbers

Un numero magico è formato concatenando i numeri 1,14,144. Per esempio 14141 è un numero magico, mentre 1444 non è un numero magico. Determinare se un numero è un magic number.

Proprietà figa: nessuna stringa è il suffix dell'altra, 1 non è il suffisso di 14, e nemmeno 14 lo è per 144, per questo gasse. E la strategia greedy? si prende sempre la stringa più grande.

17.4 Lempel-Ziv

E' un algoritmo greedy, si traversa da sinistra a destra nel testo, e voglio trovare la stringa più grande possibile che si ripete nel "passato", così da zippare meglio la stringa (è un algo di compressione).

Che si fa? scrive dei pair dove puntare e quanto sono lunghi.

In questa maniera si fa il parsing da sinistra a destra e si da gassssssssssssssssssssssssssssssss

17.5 Lexicographically Maximum Subsequence

Data una sequenza tipo **ababba** vorremmo selezionare una sequenza tipo **bab**, ma non è quella più grande, perché vogliamo quella lessicograficamente più grande infatti **aabba** < **aabc**.

abbcbccacbbcbaba

Si fa la scan da destra a sinistra e ci teniamo solo le stringhe che sono più grandi godò, nell'esempio, si prende a si prende b poi b perché è uguale a noi, poi c e tutti gli c fino in fondo e si gode, tanto vince sempre la più grande partendo da sinistra quindi gasse.

17.6 Pareto Optimal Selection

Vogliamo selezionare la parte optimal frontier, ovvero punti che prendono dominano gli altri, ovvero (x,y) domina (x_i,y_i) se $x > x_i$ e $y > y_i$,

foto proofoffffff

17.7 Woodcutter

ULTIMA LEZIONE :((((((((((((((((((((((((((((((((

una foresta di alberi, n trees, ogni albero ha la sua altezza, vogliamo massimizzare il numero di alberi che possiamo tagliare, quando tagliamo l'albero possiamo decidere se farlo cascare a destra o sinistra, ma non possiamo scegliere di tagliare un tree che casca su un'altro albero. Ovviamente vogliamo tagliare piu alberi possibile, in ogni caso anche se l'albero è stato tagliato, non possiamo tagliarlo.

FOTO PROOOOF

Semplicemente prima si controlla se l'albero si può far cascare a sinistra, se no si controlla a destra, e se può cascare ma influenza comunque stica, godo tanto è sempre un albero in meno.

17.8 queue

coda, ogni persona ha un'altezza e il numero di persone davanti a lei nella coda che sono piu alte di lei. Possiamo ricostruire la coda? in certi casi è possibile

FOTO PROF

Ovviamente si lavora al contrario, quindi da destra a sinistra. Si prende $\Theta(n^2)$ time, si sortano le altezze in decreasing order e per ognuno partendo da sinistra a destra si mette nel suo posto a patto che non distrugga la condizione di un'altro numero prima.

c'è un algoritmo piu stupido, creare un array e fare l'insert dentro all'array, ma costa quadratico ugualmente, FOTO PROF CODICE. tanto basta sempre metterlo in quella posizione visto che abbiamo sempre la

TOODOOOOOO: CERCARE UNA SOLUZIONE A QUESTO PROBLEMA PIU FIGA DELLA SKIP LIST

Soluzione con la skip list la sappiamo gia grazie Ferragina. $\log n$ di predecessor query, e n di space, godo, e per inserire un elemento li dentro????? gasse si butta il numero dove deve stare e si inserisce e via, si tira il random e si sceglie se farlo scendere e via.

17.9 Boxes

GREY + DYNAMIC GODO: n boxes, ogni box i ha w_i pesantezza e d_i surability ovvero la quantità di peso che si può mettere nel box i .

FOTO PROF

Build the highest tower with some boxes.

Assunto che sappiamo un optimal ordering dei boxes,

hint: per i primi i boxes $dp[i][k]$ = minimum weight of a tower of height K with the first i boxes. FOTO PROF con boxes e definizione di $dp[i][k]$

Per tutti i primi i boxes, voglio la minimum wight di tutte le tower di altezza k (quindi da 1 a k)

(FOTO PROF FORMULA MIN) Quindi $dp[i][k]$ sapendo tutte le soluzioni di $dp[i-1][k-1]$, allora si prende il minimo tra la torre alta quanto noi ma degli elementi di prima, oppure si prende la torre di wight minima alta meno di noi quindi $dp[i-1][k-1] + noi$, quindi w_i , ma solo se la noi possiamo sorreggere la torre. Ovviamente $\Theta(n^2)$ time.

Quindi **subproblem**: come si seleziona l'ordering migliore? tra A e B , chi si mette prima o dopo? Se A sta sotto B , allora la durabilità di A diventa $d_A = d_A - w_B$ e invece al contrario $d_B = d_B - w_A$, allora che si fa? si decide solamente in base a:

$$d_A - w_B \leq d_B - w_A \implies d_A + w_A \leq d_B + w_B$$

Kadane's Algorithm (Max Subarray Sum) Sliding Window Maximum Social Distancing (Binary search on answer)
100 Prisoners Problem (Strategia dei cicli) Duplicate Elements (Array read-only o modificabile) Floyd's Cycle Finding (Tartaruga e lepre) Majority Element (Boyer-Moore) Misra-Gries Heavy Hitter (Generalizzazione Majority) Alice and Bob (Gioco scelta numeri)

Maximum Number of Overlapping Intervals

Closest Pair of Points

Tree: Sum u (Nodi u con somma cammino = somma sottoalbero)

Tree: MPS (Max Path Sum tra due foglie)

Predecessor Problem (Gestione dinamica di un set ordinato)

Frog and Mosquitos (Intervalli dinamici e query puntuali)

Longest k-good Segment (Two Pointers)

Ilya and Queries (Query statiche su stringhe)

Little Girl and Maximum (Permutazione array per max query sum)

Number of Ways (Split array in 3 parti uguali)

Counting Inversions (con Fenwick Tree o Merge Sort)

Nested Segments (Segmenti contenuti in altri)

Update the Array (Range update, Point access con Fenwick)

Dynamic Prefix Sums with Range Update (2 Fenwick Trees)

BMQ with Occurrency (Beautiful Minimum Query)

Problema più greve (Search su prefix sums con Segment Tree)

Problema più greveeee (Successor search con BST dentro Segment Tree)

Triplets (Contare ijk tali che $A[i]A[j]A[k]$)

Range Update with Lazy Propagation

Persistent Segment Tree (Query nel passato)

Problem meno difficult (Colori distinti con occorrenza 3)

A Difficult Problem (Distinct elements occurrences)

Colored Range Query (Distinct colors in range)

Static RMQ (Range Minimum Query in $O(1)$)

Rod Cutting (Taglio asta per max profitto)

Facebook Robber (House Robber su array o DAG)

Snake and Ladders (Minimo lanci dado)

Minimum Cost Path (Grid traversal)

LCS (Longest Common Subsequence)

0/1 Knapsack (Zaino intero)

Fractional Knapsack (Zaino frazionario - Greedy)

Puzzle Exercise (Coins) (Coppie vincolate)

LIS (Longest Increasing Subsequence)

Activity Selection (Intervalli non sovrapposti)

Job Sequencing (Schedulazione con deadline)

Magic Numbers (Riconoscimento pattern 1, 14, 144)

Lexicographically Maximum Subsequence

Woodcutter (Taglio alberi ottimale)

Queue of People (Ricostruzione coda da altezze)

Boxes (Torre più alta con vincoli di peso)